CIN0136 — Desenvolvimento de Software
Bacharelado em Sistemas de Informação ▪ CIn - UFPE
Profs. Kiev Santos da Gama & Vinicius Cardoso Garcia

✏ Exercício Escolar 1A
13/10/2026

Instruções: Leia as questões com atenção e cuidado, e responda com atenção ao limite de caracteres, quando indicado.

- Esta avaliação é **individual**, **sem consulta**, e vale até **10 pontos** no total. A última questão (Q11) é **bônus** e soma por cima (sua nota pode passar de 10,0).
- A duração total é de **até 2 horas**; organize seu tempo..
- Consulta a qualquer material: **não permitida**.
- Uso de IA ou dispositivos eletrônicos: **não permitido**.
- Leia cada questão com atenção antes de responder. Gerencie bem seu tempo.
- Respostas vagas ou sem justificativa serão consideradas incompletas.
- Dúvidas podem ser expostas, **publicamente**, durante **os primeiros 30 minutos** da realização do Exercício Escolar. **Entender o enunciado** faz parte da avaliação.
- Não é permitido ir ao sanitário durante a realização do exercício, **vá antes**.

Como responder (modelo somatório).

- Cada questão tem um cenário e de 5 a 7 alternativas. Cada alternativa vale um número em potência de 2, conforme sua **posição**: a 1ª vale **2**, a 2ª **4**, a 3ª **8**, a 4ª **16**, a 5ª **32**, a 6ª **64**, a 7ª **128**.
- Marque as alternativas que julgar **verdadeiras** e escreva, no campo de resposta, a **soma** dos valores marcados.
- Correção:
  - Marcou **qualquer alternativa falsa** → a questão vale **0,0** (zera inteira).
  - Marcou **só verdadeiras, mas não todas** → nota proporcional: (nº de corretas marcadas ÷ nº total de corretas) × 1,0.
  - Marcou **todas as corretas e nenhuma falsa** → **1,0**.

Questão 1

A equipe que atende a ONG **Raízes Urbanas** recebeu, de um voluntário que é estudante de programação, um script em Python que gera automaticamente os relatórios mensais de doações. O script funciona: rodado na máquina do voluntário, produz o relatório correto em segundos. O voluntário mencionou que levou só duas tardes para escrevê-lo. A coordenadora da ONG gostou do resultado e pediu que a equipe incorporasse essa geração de relatórios ao sistema que estão construindo, que usa Node.js. Ao abrir o script, a equipe encontra um único arquivo de 300 linhas, sem nenhum teste, sem comentários, com todas as regras de cálculo e o acesso ao banco misturados na mesma função. O voluntário se formou e não está mais disponível para explicar o código.

Assinale as verdadeiras:

- [ ] 1ª (2): O fato de o script ter sido escrito em Python, e não em Node.js como o resto do sistema, é a principal razão pela qual ele não pode ser considerado um artefato de qualidade para o projeto, pois compromete a integração técnica.
- [ ] 2ª (4): Que o script "funcione" e produza o relatório correto é insuficiente para classificá-lo como software de qualidade, porque atributos como manutenibilidade e testabilidade não estão presentes — e são justamente eles que permitirão a evolução do relatório no futuro.
- [ ] 3ª (8): A dificuldade que a equipe terá para modificar o script, agravada pela saída do voluntário, é um desafio de natureza tanto técnica (código sem estrutura) quanto organizacional (perda de conhecimento e ausência de documentação).
- [ ] 4ª (16): Incorporar o script no estado atual permitiria entregar valor à coordenadora rapidamente; como o comportamento já foi validado na prática, o esforço de reorganizá-lo antes da incorporação seria uma otimização prematura neste momento do projeto.
- [ ] 5ª (32): Como a saída do relatório já foi conferida pela coordenadora e está correta, a incorporação pode prosseguir sem testes automatizados por ora, ficando a cobertura de testes como uma melhoria de qualidade desejável, mas não relacionada ao risco de a lógica quebrar em alterações futuras.

Resposta (soma): _____

Questão 2

Numa aula de revisão, o professor propõe o seguinte caso para discussão. Uma equipe precisa construir um aplicativo de cadastro de beneficiários para uma ONG. A equipe tem cinco integrantes, um dos quais nunca havia trabalhado com banco de dados antes. Duas abordagens são colocadas na mesa. Na **Abordagem 1**, a equipe entrega na primeira semana apenas o formulário de nome; na segunda, acrescenta o formulário de endereço; na terceira, o histórico de atendimentos (cada entrega adiciona uma parte nova e completa do sistema). Na **Abordagem 2**, a equipe entrega na primeira semana uma versão do sistema inteiro com os três campos, porém simples, feia e sem validação; nas semanas seguintes, refina essa mesma versão, adicionando validação, melhorando a interface e ajustando com base no feedback. A coordenadora da ONG só poderá dar feedback a partir da terceira semana, por estar viajando.

Assinale as verdadeiras:

- [ ] 1ª (2): A Abordagem 1 descreve um modelo incremental, pois o produto cresce por adição de partes novas e completas a cada entrega, enquanto a Abordagem 2 descreve um modelo iterativo, em que a mesma versão é refinada sucessivamente.
- [ ] 2ª (4): Ambas as abordagens entregam software ao longo do tempo em vez de tudo no final, o que as diferencia do modelo cascata clássico, no qual a validação do produto pelo usuário tende a ocorrer tardiamente.
- [ ] 3ª (8): A Abordagem 2 é preferível à Abordagem 1 em qualquer projeto, porque refinar uma versão existente é sempre menos arriscado do que adicionar módulos novos, que podem quebrar o que já funciona.

- [ ] 4ª (16): Como a Abordagem 2 entrega uma versão do sistema inteiro logo na primeira semana, ela corresponde ao modelo cascata, já que o cascata também busca ter o sistema completo o quanto antes.
- [ ] 5ª (32): No modelo iterativo, cada ciclo assume explicitamente que a equipe vai aprender algo e que o produto vai mudar, o que torna o feedback (quando disponível) uma entrada planejada do processo, e não uma interrupção dele.
- [ ] 6ª (64): O modelo cascata foi um erro histórico da engenharia de software e não possui nenhum contexto de aplicação legítimo nos dias de hoje, tendo sido inteiramente substituído pelos modelos incremental e iterativo.

Resposta (soma): _____

Questão 3

A equipe que atende a ONG **Raízes Urbanas** está encerrando a terceira Sprint. Nesta Sprint, entregaram o módulo de cadastro de voluntários e corrigiram três bugs reportados pela coordenadora na Sprint anterior. Durante a semana houve um desentendimento entre dois integrantes sobre qual editor de código a equipe deveria padronizar, o que gerou certo mal-estar. A equipe adotou Scrum com Sprints de duas semanas e usa um board no GitHub Projects para visualizar o fluxo de trabalho. A coordenadora confirmou presença no evento de fechamento da Sprint. O integrante responsável por guardar e priorizar o backlog percebeu que o board acumulou muitos itens simultaneamente na coluna "Em progresso".

Assinale as verdadeiras:

- [ ] 1ª (2): O evento de Sprint Review, ao final da Sprint, serve para demonstrar o incremento à coordenadora e coletar seu feedback sobre o que foi construído, enquanto a reflexão da equipe sobre o próprio processo de trabalho pertence à Retrospectiva.
- [ ] 2ª (4): Como a coordenadora estará presente no fechamento da Sprint, a equipe deve aproveitar esse momento para, além de apresentar o incremento, resolver ali o desentendimento sobre o editor de código, usando a presença de uma figura externa para mediar o conflito interno.
- [ ] 3ª (8): O acúmulo de muitos itens simultâneos em "Em progresso" contraria a lógica de limitar o trabalho em progresso (WIP); reduzir os itens em andamento tende a diminuir o cycle time, acelerando a entrega efetiva de cada item.
- [ ] 4ª (16): O papel de guardar e priorizar o backlog, exercido pelo integrante mencionado, corresponde ao do Product Owner, responsável por representar os interesses do stakeholder na ordenação do que será construído.
- [ ] 5ª (32): Por usarem um board de visualização de fluxo como o do Kanban, a equipe está, na prática, abrindo mão dos Sprints e do compromisso de entrega ao final de cada ciclo, já que board Kanban e Sprints do Scrum são mutuamente exclusivos.

- [ ] 6ª (64): O Daily Standup tem como propósito sincronizar a equipe em torno do que foi feito, do que será feito e dos impedimentos, e não substitui a Sprint Review, cujo foco é a inspeção do incremento com o stakeholder.

Resposta (soma): _____

Questão 4

A equipe está levantando requisitos do **PetFood**, o sistema de gestão de produtos para pet shop. O dono do pet shop mencionou que pretende, no futuro, abrir uma segunda loja em outro bairro. Durante uma visita, a equipe observou a atendente trabalhando: ela consultava os preços num sistema antigo, mas mantinha ao lado um caderno onde anotava quais clientes costumavam comprar ração de marcas específicas< informação que o sistema atual não guardava. A atendente comentou que o caderno era um presente de aniversário. Em conversa posterior, o dono afirmou que o sistema "precisa ser rápido" e que "qualquer funcionário novo tem que conseguir usar sem treinamento". A equipe também registrou que o sistema deve permitir cadastrar produtos com nome, preço e categoria.

Assinale as verdadeiras:

- [ ] 1ª (2): "Cadastrar produtos com nome, preço e categoria" e "usar o sistema sem treinamento" são requisitos de naturezas distintas: o primeiro descreve uma função verificável por presença ou ausência, o segundo é um atributo de qualidade cuja verificação depende de definir o que conta como "sem treinamento".
- [ ] 2ª (4): A exigência de que "qualquer funcionário novo consiga usar sem treinamento" é um requisito funcional, já que se refere diretamente ao uso do sistema por um usuário.
- [ ] 3ª (8): A descoberta do caderno pela observação da atendente em seu ambiente real cumpre um papel que a entrevista com o dono dificilmente cumpriria sozinha: expor uma necessidade que ninguém verbaliza porque, para quem trabalha ali, ela já é parte invisível da rotina.
- [ ] 4ª (16): A afirmação de que o sistema "precisa ser rápido", tal como o dono a expressou, já pode ser levada diretamente para a fase de testes e verificada de forma objetiva, sem necessidade de trabalho adicional de refinamento com o cliente.
- [ ] 5ª (32): O caderno indica um requisito que o sistema atual não atende, e o fato de ele ter surgido da observação, e não da fala do dono, ilustra por que restringir a elicitação a perguntar "o que você quer?" tende a produzir um sistema que ignora parte do trabalho real.

Resposta (soma): _____

Questão 5

A equipe que atende a ONG **Raízes Urbanas** está refinando o backlog antes da sessão com a coordenadora. A reunião foi remarcada duas vezes por causa de um feriado municipal. Entre os itens levantados, aparecem estes três: **(I)** "Como coordenadora, quero gerenciar voluntários, doações, eventos e relatórios para administrar a ONG"; **(II)** "Como voluntária, quero redefinir minha senha para recuperar o acesso à minha conta quando

esquecê-la"; **(III)** "Como desenvolvedor, quero migrar o banco para PostgreSQL para melhorar a performance". A equipe discutiu por dez minutos se usaria Trello ou GitHub Projects para o board. A coordenadora sinalizou que classificaria "quase tudo" como prioridade máxima na hora de priorizar.

Assinale as verdadeiras:

- [ ] 1ª (2): O item (I) viola o critério *Small* do INVEST e, por reunir quatro áreas administrativas distintas, comporta-se mais como um Epic do que como uma user story, o que sugere quebrá-lo em stories menores antes de estimá-lo.
- [ ] 2ª (4): O item (III) é uma boa user story porque segue corretamente o formato "Como [persona], quero [ação], para [benefício]", e o formato correto é o que garante que a story entregue valor ao usuário.
- [ ] 3ª (8): Para o item (II), um critério de aceitação adequado em Gherkin seria: "Dado que a voluntária solicitou a redefinição, Quando ela abre o link após o prazo de validade, Então vê uma mensagem de erro e pode solicitar um novo link", um cenário de caminho triste tão legítimo quanto os de sucesso.
- [ ] 4ª (16): O item (III), embora bem formado quanto ao formato, falha no critério *Valuable* do INVEST: seu benefício está descrito da perspectiva técnica da equipe, não de um resultado percebido por quem usa o sistema, o que costuma indicar uma tarefa a ser absorvida por outra story, e não um item de backlog de produto por si só.
- [ ] 5ª (32): Diante de a coordenadora querer classificar "quase tudo" como prioridade máxima, a saída correta em MoSCoW é registrar a maior parte dos itens como *Must Have*, já que *Must* é justamente a categoria que acomoda tudo aquilo que o cliente considera importante.
- [ ] 6ª (64): Marcar o item (I) como *Won't Have (now)* significaria descartá-lo definitivamente do produto, razão pela qual essa classificação deve ser evitada para funcionalidades administrativas que a ONG claramente vai precisar.
- [ ] 7ª (128): O item (II) já está pronto como está e não precisa de critérios de aceitação, pois seu benefício ("recuperar o acesso") é claro o suficiente para que a equipe saiba, sem ambiguidade, quando a story foi concluída.

Resposta (soma): _____

Questão 6

A equipe está no fim do Sprint 1 do **PetFood**. A funcionalidade de busca de produtos está pronta e passa em todos os critérios de aceitação de funcionalidade que a equipe escreveu. A busca foi implementada por dois integrantes que preferiram programar juntos no mesmo computador. Ao testar com a base real do pet shop, com milhares de produtos, notaram que a busca leva cerca de 8 segundos para retornar. O pet shop fica num bairro onde a internet costuma oscilar à tarde. Além disso, um integrante comentou que o código da busca "funciona, mas está tão embolado que, se precisarem mudar a regra de ordenação depois, vai ser um pesadelo". A equipe vai definir seu Definition of Done antes do Sprint 2.

Assinale as verdadeiras:

- ☐ 1ª (2): Embora a busca provavelmente responda rápido durante os testes da equipe, feitos com poucos produtos, os 8 segundos sob a base real mostram que ela cumpre a funcionalidade mas falha em desempenho, dois atributos distintos, e passar num não garante o outro.
- ☐ 2ª (4): O código "embolado" que dificultaria mudar a ordenação é um problema de manutenibilidade; e ainda que o usuário final nunca perceba a organização interna do código, isso não torna o problema menos real para a equipe, que arcará com o custo na evolução do sistema.
- ☐ 3ª (8): Como a lentidão de 8 segundos pode ser causada pela oscilação da internet do bairro à tarde, trata-se de um problema de infraestrutura externa, e não de um atributo de qualidade do sistema que a equipe precise endereçar.
- ☐ 4ª (16): Incluir no Definition of Done um critério como "toda funcionalidade nova deve ter o tempo de resposta verificado sob carga realista" operacionaliza qualidade de forma contínua, e o fato de o DoD anterior da equipe não conter nenhum critério de desempenho é justamente o que permitiu que esse problema só aparecesse agora, perto do fim do Sprint.
- ☐ 5ª (32): Uma vez que a busca já passa em todos os critérios de aceitação escritos pela equipe, considerá-la "pronta" é a decisão correta; ajustes de desempenho e organização do código, se necessários, são melhorias que pertencem a um momento posterior à conclusão da funcionalidade.
- ☐ 6ª (64): A melhor forma de garantir qualidade neste caso seria adiar qualquer preocupação com desempenho e manutenibilidade até uma fase final de verificação, quando todas as funcionalidades estiverem prontas e puderem ser avaliadas de uma só vez.

Resposta (soma): _____

Questão 7

Na aula de escopo assistido por IA, a equipe que atende a ONG **Raízes Urbanas** seguiu o roteiro proposto: primeiro, cada integrante escreveu sozinho, sem abrir a IA, o que entendia ser o escopo essencial do caso de treino; só depois compararam entre si e recorreram ao assistente. Um dos integrantes chegou atrasado e teve cinco minutos a menos para escrever sua versão. As versões individuais divergiam bastante entre si. Em seguida, deram à IA o papel de analista cético e pediram que ela apontasse personas esquecidas e o "problema por trás do problema". A sala estava com o ar-condicionado quebrado naquele dia. O escopo do MVP do projeto real já fora aprovado pela coordenadora em sprint anterior.

Assinale as verdadeiras:

- ☐ 1ª (2): O roteiro teria sido mais eficiente se a equipe tivesse aberto a IA primeiro, deixado que ela propusesse uma visão de escopo, e só depois pedido que cada integrante divergisse a partir dela, partindo de uma base estruturada em vez de versões cruas e conflitantes.

- [ ] 2ª (4): Dar à IA o papel de analista cético que questiona antes de gerar, em vez de pedir que ela defina o escopo, protege a equipe contra o viés de confirmação.
- [ ] 3ª (8): Uma vez que a IA é boa em revelar personas esquecidas, faz sentido deixar que ela própria decida quais itens entram na fronteira do MVP, contanto que a equipe valide a lista no final.
- [ ] 4ª (16): Se a equipe não consegue explicar, sem o assistente, por que cada história está no backlog e que valor entrega, então, pela política da disciplina, essas histórias ainda não são propriamente da equipe.
- [ ] 5ª (32): O uso adequado da IA sobre o escopo já aprovado seria pedir que ela reescrevesse o Canvas de Visão do zero com olhar cético, aproveitando que costuma produzir uma versão mais completa que a montada na correria da Semana 2.
- [ ] 6ª (64): Ainda que a IA cética tenha feito boas perguntas, isso não equivale a ter elicitado a necessidade real: as lacunas que ela expõe entram no projeto como perguntas a levar à coordenadora, não como requisitos que a equipe já pode dar por levantados.
- [ ] 7ª (128): Manter a divergência individual antes de recorrer à IA foi o que preservou a informação mais útil do exercício: o desacordo entre as visões, que some quando todos partem de uma mesma resposta pronta apresentada pela ferramenta.

Resposta (soma): _____

Questão 8

No **PetFood**, o frontend em React exibe a lista de produtos disponíveis; o backend em Express expõe a rota `GET /api/produtos`; os dados ficam num banco PostgreSQL. A equipe hesitou entre chamar a rota de `/api/produtos` ou `/api/products` e acabou padronizando em português. Um integrante colocou, dentro do controller dessa rota, tanto a montagem da resposta HTTP quanto a regra de negócio "só retornar produtos com estoque maior que zero" quanto o comando SQL que acessa o banco. Esse integrante é o único da equipe que já tinha mexido com SQL antes. Durante a Sprint Review, a coordenadora pediu que, no futuro, produtos esgotados também aparecessem, porém marcados como indisponíveis.

Assinale as verdadeiras:

- [ ] 1ª (2): Para atender o novo pedido da coordenadora (mostrar esgotados como indisponíveis), a equipe precisará alterar a regra de filtragem que hoje vive no controller; se essa regra estivesse isolada numa camada de serviço, a rota `GET /api/produtos` e o código de resposta HTTP poderiam permanecer intocados.
- [ ] 2ª (4): Se o time de frontend precisar exibir os produtos esgotados como indisponíveis, terá de esperar que o backend altere o comando SQL interno, pois é o formato do SQL que o contrato da rota `GET /api/produtos` expõe ao cliente.
- [ ] 3ª (8): Com a regra de estoque, o SQL e a resposta HTTP no mesmo controller, um colega que quisesse alterar apenas o formato do JSON de resposta precisaria abrir e entender também a regra de negócio e a consulta ao banco, mesmo sem ter intenção de mexer em nenhuma das duas.

- [ ] 4ª (16): Se a equipe extraísse o comando SQL para uma camada de repositório, o controller passaria a depender da interface desse repositório em vez do SQL concreto; assim, trocar PostgreSQL por outro banco tenderia a afetar o repositório, e não o controller nem a regra de negócio.
- [ ] 5ª (32): Como o React roda no navegador do usuário e o Express num servidor, os dois estão em processos separados; logo, a regra "estoque maior que zero" poderia ser implementada tanto no React quanto no Express sem diferença arquitetural, já que ambos são código JavaScript da mesma equipe.

Resposta (soma): _____

Questão 9

Na equipe da ONG **Raízes Urbanas**, dois integrantes trabalham no mesmo arquivo `voluntarioController.js`. Os dois combinaram de programar em horários alternados para não se atrapalhar, um de manhã e outro à noite. Esse arquivo, com cerca de 250 linhas, contém a validação de CPF, o envio do e-mail de boas-vindas, o acesso ao banco e a montagem da resposta HTTP, tudo em uma função chamada `handle()`. Ao tentar fazer merge das duas contribuições no fim da semana, o GitHub acusou vários conflitos no mesmo trecho. Um dos integrantes usa tema escuro no editor e o outro tema claro. Um deles precisou mudar só o texto do e-mail de boas-vindas e, para isso, teve de abrir e reler a função inteira.

Assinale as verdadeiras:

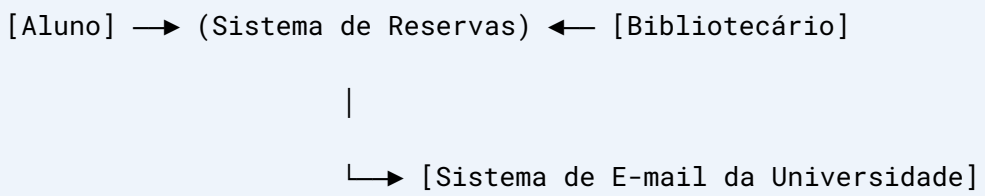
- [ ] 1ª (2): Enquanto a montagem da resposta HTTP, a validação de CPF e a consulta ao banco compartilharem o corpo de `handle()`, uma correção no texto do e-mail e uma correção na regra de validação, feitas por pessoas diferentes na mesma semana, tendem a recair sobre linhas vizinhas do mesmo bloco, e é aí que o merge acusa conflito.
- [ ] 2ª (4): Se os dois integrantes passassem a trabalhar no mesmo turno e a integrar várias vezes ao dia, os conflitos de merge neste arquivo deixariam de ocorrer, pois estariam sempre partindo da versão mais recente um do outro antes de alterar qualquer parte de `handle()`.
- [ ] 3ª (8): Extrair a validação de CPF para uma função própria permitiria escrever um teste que verifica se um CPF malformado é rejeitado, sem que esse teste dispare o envio de e-mail nem precise de uma conexão com o banco para rodar.
- [ ] 4ª (16): Renomear `handle()` para `handleCadastroVoluntario()` tornaria a intenção do código mais evidente para quem o lê pela primeira vez e, por deixar o propósito explícito no nome, seria suficiente para resolver a dificuldade que a equipe tem de alterar esse arquivo sem efeitos colaterais.
- [ ] 5ª (32): Se o envio do e-mail passasse para uma função `enviarEmailBoasVindas(dados)` chamada por `handle()`, uma futura troca do serviço de e-mail, do modo atual para um provedor com API diferente, poderia ser feita alterando apenas essa função, sem reabrir a lógica de validação nem a de resposta.

- [ ] 6ª (64): Quebrar as 250 linhas em dois arquivos de cerca de 125, deixando em cada um um pedaço da validação, um pedaço do e-mail e um pedaço do acesso ao banco misturados, reduziria o número de linhas por arquivo, mas dois desenvolvedores mexendo em responsabilidades distintas ainda poderiam colidir, agora espalhados por dois arquivos em vez de um.

Resposta (soma): _____

Questão 10

Equipe de 4 pessoas, 4 meses, para construir um sistema de reservas de uma biblioteca universitária, hoje um monólito Client/Server (React + Express em camadas + um banco PostgreSQL). Na Sprint 1 Review, a equipe levou este diagrama:



E registrou no repositório:

> ***ADR-003 — Banco único*** *Contexto: equipe de 4, prazo de 4 meses, sem experiência prévia em operar serviços distribuídos. Decisão: manter um único banco PostgreSQL para todo o sistema. Consequências: simplifica deploy e desenvolvimento agora; se reservas e catálogo crescerem muito, pode ser necessário revisitar.*

Dois integrantes queriam microsserviços porque viram num vídeo que "é o que as big techs usam". O bibliotecário perguntou se haveria app de celular, o que não está no escopo.

Assinale as verdadeiras:

- [ ] 1ª (2): Dois integrantes defendem separar reservas e catálogo em bancos distintos agora, argumentando que o *Bounded Context* prega que cada serviço seja dono do seu dado e que, portanto, o banco único já é uma violação arquitetural a corrigir antes que o acoplamento cresça. Seguir esse argumento seria um erro: o Bounded Context governa *como* delimitar serviços que a equipe decidiu ter, e não é razão, por si, para introduzir a separação de serviços num sistema cujo contexto (4 pessoas, 4 meses) não a justifica.
- [ ] 2ª (4): Alguém sugere apagar o campo *Consequências* do ADR-003 e deixá-lo só com Contexto e Decisão, sob o argumento de que, como a decisão de banco único claramente foi acertada para o momento, registrar uma possível revisão futura só abre margem para questionamentos desnecessários. Essa edição enfraqueceria o ADR: é justamente o registro do custo e da condição de revisão que impede a equipe futura de tratar a escolha como definitiva e de refazer a análise do zero quando o contexto mudar.

- [ ] 3ª (8): Para a próxima Review, a equipe quer mostrar ao bibliotecário como React, Express e banco se distribuem e se comunicam. Um integrante propõe apenas anotar as tecnologias sobre as caixas do diagrama atual, mantendo os mesmos elementos, e conclui que isso já o converte num diagrama de Contêiner do C4, pronto para essa finalidade. A proposta é adequada, pois é a indicação das tecnologias sobre os elementos existentes que caracteriza a passagem do Nível de Contexto para o Nível de Contêiner.
- [ ] 4ª (16): Como o bibliotecário, na Review, não conseguiu opinar sobre decisões técnicas e ainda perguntou sobre um app de celular fora do escopo, um integrante conclui que o diagrama de Contexto foi inadequado para aquela audiência e que teria sido melhor levar o diagrama de Contêiner, mais detalhado, para dar a ele base técnica de avaliação.
- [ ] 5ª (32): Os defensores dos microsserviços argumentam que, como o próprio ADR-003 admite que a separação pode ser necessária se reservas e catálogo crescerem, o mais eficiente seria já nascer com os serviços separados, evitando o custo de migrar um monólito depois, afinal, refatorar de monólito para microsserviços é sabidamente caro. Esse raciocínio deve ser recusado no contexto dado: antecipar a separação impõe hoje, a uma equipe sem experiência distribuída e com 4 meses, um custo operacional certo para evitar um custo futuro que é apenas possível e condicional ao crescimento.
- [ ] 6ª (64): Um integrante afirma que registrar o ADR-003 foi esforço perdido: como a arquitetura provavelmente vai mudar ao longo do semestre, documentar agora uma decisão que pode ser revista é trabalho que será jogado fora assim que a separação de bancos acontecer. Esse argumento não se sustenta: quando a separação ocorrer, um novo ADR registrará a mudança, e o ADR-003 seguirá explicando em que contexto o banco único fez sentido, de modo que o registro atual não é descartado, e sim torna-se o histórico que evita rediscutir a escolha original.
- [ ] 7ª (128): Diante da insistência nos microsserviços, um integrante propõe encerrar a discussão declarando que o monólito atual é uma escolha "sem trade-offs" para este projeto, já que não se enxerga nenhuma desvantagem nele no prazo de 4 meses, e que isso prova objetivamente que é a decisão certa.

Resposta (soma): _____

Questão 11 - bônus

Na equipe da ONG **Raízes Urbanas**, quatro pessoas trabalham no mesmo repositório. Durante a Sprint, combinaram que cada funcionalidade seria desenvolvida em um branch próprio e integrada à `main` via Pull Request revisado por outro colega. Dois integrantes preferem fazer commits no fim do dia; os outros dois, a cada poucas horas. Numa dada semana: a integrante A abriu um branch para o cadastro de doações e passou seis dias sem integrá-lo, acumulando muitas mudanças; o integrante B abriu um PR pequeno e pediu revisão, mas ninguém revisou por três dias; e a dupla C/D, trabalhando no mesmo arquivo em branches diferentes, adiou a integração para o último dia. A `main` está protegida contra push direto. Ao final, houve conflitos grandes e demorados de resolver.

Assinale as verdadeiras:

- [ ] 1ª (2): Como o objetivo de um PR é dar a outro colega a chance de revisar antes da integração, e o PR de B ficou três dias parado sem revisor, a equipe deveria concluir que, quando não há disponibilidade de revisão em tempo hábil, o mais alinhado ao propósito da revisão é autorizar o próprio autor a aprovar e integrar o seu PR, para que o fluxo não trave, a revisão que não acontece a tempo perde a função.
- [ ] 2ª (4): Considerando que a base da `main` evolui enquanto um branch permanece isolado, o branch de seis dias de A e a integração de última hora de C/D compartilham a mesma causa de fundo: ambos deixaram a distância entre o seu trabalho e a `main` crescer sem reconciliação, de modo que o esforço de merge não foi criado no dia do conflito, mas acumulado silenciosamente nos dias anteriores em que a integração foi adiada.
- [ ] 3ª (8): Já que o conflito concreto surgiu entre C e D por editarem o mesmo arquivo em paralelo, e como o Git resolve automaticamente merges quando as pessoas trabalham em arquivos diferentes, a lição consistente com o funcionamento do Git é organizar a divisão de tarefas de modo que dois integrantes nunca fiquem responsáveis por um mesmo arquivo na mesma Sprint, eliminando na origem a possibilidade de conflito.
- [ ] 4ª (16): Uma vez que a `main` está protegida contra push direto, e que essa proteção obriga toda mudança a passar por um branch e um PR antes de chegar à linha principal, é coerente concluir que foi essa exigência de intermediação que represou as mudanças em branches separados por vários dias, logo, afrouxar a proteção para permitir push direto em casos urgentes reduziria o represamento e, com ele, os conflitos.
- [ ] 5ª (32): Como dois integrantes commitam só no fim do dia e os outros dois a cada poucas horas, e sabendo que integrações mais frequentes tendem a produzir conflitos menores, a conclusão coerente é que padronizar todos para commitar apenas uma vez ao dia alinharia a cadência da equipe e, com uma única janela diária de integração, concentraria e reduziria os conflitos.

Resposta (soma): _____

CIN0136 — Desenvolvimento de Software

Bacharelado em Sistemas de Informação ▪ CIn - UFPE

Profs. Kiev Santos da Gama & Vinicius Cardoso Garcia

✏ Exercício Escolar 1B

13/10/2026

Instruções: Leia as questões com atenção e cuidado, e responda com atenção ao limite de caracteres, quando indicado.

- Esta avaliação é **individual**, **sem consulta**, e vale até **10 pontos** no total. A última questão (Q11) é **bônus** e soma por cima (sua nota pode passar de 10,0).
- A duração total é de **até 2 horas**; organize seu tempo..
- Consulta a qualquer material: **não permitida**.
- Uso de IA ou dispositivos eletrônicos: **não permitido**.
- Leia cada questão com atenção antes de responder. Gerencie bem seu tempo.
- Respostas vagas ou sem justificativa serão consideradas incompletas.
- Dúvidas podem ser expostas, **publicamente**, durante **os primeiros 30 minutos** da realização do Exercício Escolar. **Entender o enunciado** faz parte da avaliação.
- Não é permitido ir ao sanitário durante a realização do exercício, **vá antes**.

Como responder (modelo somatório).

- Cada questão tem um cenário e de 5 a 7 alternativas. Cada alternativa vale um número em potência de 2, conforme sua **posição**: a 1ª vale **2**, a 2ª **4**, a 3ª **8**, a 4ª **16**, a 5ª **32**, a 6ª **64**, a 7ª **128**.
- Marque as alternativas que julgar **verdadeiras** e escreva, no campo de resposta, a **soma** dos valores marcados.
- Correção:
  - Marcou **qualquer alternativa falsa** → a questão vale **0,0** (zera inteira).
  - Marcou **só verdadeiras, mas não todas** → nota proporcional: (nº de corretas marcadas ÷ nº total de corretas) × 1,0.
  - Marcou **todas as corretas e nenhuma falsa** → **1,0**.

Questão 1

A equipe que atende a ONG **Raízes Urbanas** recebeu, de um voluntário que é estudante de programação, um script em Python que gera automaticamente os relatórios mensais de doações. O script funciona: rodado na máquina do voluntário, produz o relatório correto em segundos. O voluntário mencionou que levou só duas tardes para escrevê-lo. A coordenadora da ONG gostou do resultado e pediu que a equipe incorporasse essa geração de relatórios ao sistema que estão construindo, que usa Node.js. Ao abrir o script, a equipe encontra um único arquivo de 300 linhas, sem nenhum teste, sem comentários, com todas as regras de cálculo e o acesso ao banco misturados na mesma função. O voluntário se formou e não está mais disponível para explicar o código.

Assinale as verdadeiras:

- [ ] 1ª (2): A dificuldade que a equipe terá para modificar o script, agravada pela saída do voluntário, é um desafio de natureza tanto técnica (código sem estrutura) quanto organizacional (perda de conhecimento e ausência de documentação).
- [ ] 2ª (4): O fato de o script ter sido escrito em Python, e não em Node.js como o resto do sistema, é a principal razão pela qual ele não pode ser considerado um artefato de qualidade para o projeto, pois compromete a integração técnica.
- [ ] 3ª (8): Como a saída do relatório já foi conferida pela coordenadora e está correta, a incorporação pode prosseguir sem testes automatizados por ora, ficando a cobertura de testes como uma melhoria de qualidade desejável, mas não relacionada ao risco de a lógica quebrar em alterações futuras.
- [ ] 4ª (16): Que o script "funcione" e produza o relatório correto é insuficiente para classificá-lo como software de qualidade, porque atributos como manutenibilidade e testabilidade não estão presentes, e são justamente eles que permitirão a evolução do relatório no futuro.
- [ ] 5ª (32): Incorporar o script no estado atual permitiria entregar valor à coordenadora rapidamente; como o comportamento já foi validado na prática, o esforço de reorganizá-lo antes da incorporação seria uma otimização prematura neste momento do projeto.

Resposta (soma): _____

Questão 2

Numa aula de revisão, o professor propõe o seguinte caso para discussão. Uma equipe precisa construir um aplicativo de cadastro de beneficiários para uma ONG. A equipe tem cinco integrantes, um dos quais nunca havia trabalhado com banco de dados antes. Duas abordagens são colocadas na mesa. Na **Abordagem 1**, a equipe entrega na primeira semana apenas o formulário de nome; na segunda, acrescenta o formulário de endereço; na terceira, o histórico de atendimentos (cada entrega adiciona uma parte nova e completa do sistema). Na **Abordagem 2**, a equipe entrega na primeira semana uma versão do sistema inteiro com os três campos, porém simples, feia e sem validação; nas semanas seguintes, refina essa mesma versão, adicionando validação, melhorando a interface e ajustando com base no feedback. A coordenadora da ONG só poderá dar feedback a partir da terceira semana, por estar viajando.

Assinale as verdadeiras:

- [ ] 1ª (2): O modelo cascata foi um erro histórico da engenharia de software e não possui nenhum contexto de aplicação legítimo nos dias de hoje, tendo sido inteiramente substituído pelos modelos incremental e iterativo.
- [ ] 2ª (4): A Abordagem 1 descreve um modelo incremental, pois o produto cresce por adição de partes novas e completas a cada entrega, enquanto a Abordagem 2 descreve um modelo iterativo, em que a mesma versão é refinada sucessivamente.
- [ ] 3ª (8): Como a Abordagem 2 entrega uma versão do sistema inteiro logo na primeira semana, ela corresponde ao modelo cascata, já que o cascata também busca ter o sistema completo o quanto antes.

- [ ] 4ª (16): No modelo iterativo, cada ciclo assume explicitamente que a equipe vai aprender algo e que o produto vai mudar, o que torna o feedback (quando disponível) uma entrada planejada do processo, e não uma interrupção dele.
- [ ] 5ª (32): A Abordagem 2 é preferível à Abordagem 1 em qualquer projeto, porque refinar uma versão existente é sempre menos arriscado do que adicionar módulos novos, que podem quebrar o que já funciona.
- [ ] 6ª (64): Ambas as abordagens entregam software ao longo do tempo em vez de tudo no final, o que as diferencia do modelo cascata clássico, no qual a validação do produto pelo usuário tende a ocorrer tardiamente.

Resposta (soma): _____

Questão 3

A equipe que atende a ONG **Raízes Urbanas** está encerrando a terceira Sprint. Nesta Sprint, entregaram o módulo de cadastro de voluntários e corrigiram três bugs reportados pela coordenadora na Sprint anterior. Durante a semana houve um desentendimento entre dois integrantes sobre qual editor de código a equipe deveria padronizar, o que gerou certo mal-estar. A equipe adotou Scrum com Sprints de duas semanas e usa um board no GitHub Projects para visualizar o fluxo de trabalho. A coordenadora confirmou presença no evento de fechamento da Sprint. O integrante responsável por guardar e priorizar o backlog percebeu que o board acumulou muitos itens simultaneamente na coluna "Em progresso".

Assinale as verdadeiras:

- [ ] 1ª (2): Por usarem um board de visualização de fluxo como o do Kanban, a equipe está, na prática, abrindo mão dos Sprints e do compromisso de entrega ao final de cada ciclo, já que board Kanban e Sprints do Scrum são mutuamente exclusivos.
- [ ] 2ª (4): O evento de Sprint Review, ao final da Sprint, serve para demonstrar o incremento à coordenadora e coletar seu feedback sobre o que foi construído, enquanto a reflexão da equipe sobre o próprio processo de trabalho pertence à Retrospectiva.
- [ ] 3ª (8): O Daily Standup tem como propósito sincronizar a equipe em torno do que foi feito, do que será feito e dos impedimentos, e não substitui a Sprint Review, cujo foco é a inspeção do incremento com o stakeholder.
- [ ] 4ª (16): Como a coordenadora estará presente no fechamento da Sprint, a equipe deve aproveitar esse momento para, além de apresentar o incremento, resolver ali o desentendimento sobre o editor de código, usando a presença de uma figura externa para mediar o conflito interno.
- [ ] 5ª (32): O acúmulo de muitos itens simultâneos em "Em progresso" contraria a lógica de limitar o trabalho em progresso (WIP); reduzir os itens em andamento tende a diminuir o cycle time, acelerando a entrega efetiva de cada item.
- [ ] 6ª (64): O papel de guardar e priorizar o backlog, exercido pelo integrante mencionado, corresponde ao do Product Owner, responsável por representar os interesses do stakeholder na ordenação do que será construído.

Resposta (soma): _____

Questão 4

A equipe está levantando requisitos do **PetFood**, o sistema de gestão de produtos para pet shop. O dono do pet shop mencionou que pretende, no futuro, abrir uma segunda loja em outro bairro. Durante uma visita, a equipe observou a atendente trabalhando: ela consultava os preços num sistema antigo, mas mantinha ao lado um caderno onde anotava quais clientes costumavam comprar ração de marcas específicas, informação que o sistema atual não guardava. A atendente comentou que o caderno era um presente de aniversário. Em conversa posterior, o dono afirmou que o sistema "precisa ser rápido" e que "qualquer funcionário novo tem que conseguir usar sem treinamento". A equipe também registrou que o sistema deve permitir cadastrar produtos com nome, preço e categoria.

Assinale as verdadeiras:

- ☐ 1ª (2): A afirmação de que o sistema "precisa ser rápido", tal como o dono a expressou, já pode ser levada diretamente para a fase de testes e verificada de forma objetiva, sem necessidade de trabalho adicional de refinamento com o cliente.
- ☐ 2ª (4): "Cadastrar produtos com nome, preço e categoria" e "usar o sistema sem treinamento" são requisitos de naturezas distintas: o primeiro descreve uma função verificável por presença ou ausência, o segundo é um atributo de qualidade cuja verificação depende de definir o que conta como "sem treinamento".
- ☐ 3ª (8): A exigência de que "qualquer funcionário novo consiga usar sem treinamento" é um requisito funcional, já que se refere diretamente ao uso do sistema por um usuário.
- ☐ 4ª (16): O caderno indica um requisito que o sistema atual não atende, e o fato de ele ter surgido da observação, e não da fala do dono, ilustra por que restringir a elicitação a perguntar "o que você quer?" tende a produzir um sistema que ignora parte do trabalho real.
- ☐ 5ª (32): A descoberta do caderno pela observação da atendente em seu ambiente real cumpre um papel que a entrevista com o dono dificilmente cumpriria sozinha: expor uma necessidade que ninguém verbaliza porque, para quem trabalha ali, ela já é parte invisível da rotina.

Resposta (soma): _____

Questão 5

A equipe que atende a ONG **Raízes Urbanas** está refinando o backlog antes da sessão com a coordenadora. A reunião foi remarcada duas vezes por causa de um feriado municipal. Entre os itens levantados, aparecem estes três: **(I)** "Como coordenadora, quero gerenciar voluntários, doações, eventos e relatórios para administrar a ONG"; **(II)** "Como voluntária, quero redefinir minha senha para recuperar o acesso à minha conta quando esquecê-la"; **(III)** "Como desenvolvedor, quero migrar o banco para PostgreSQL para

melhorar a performance". A equipe discutiu por dez minutos se usaria Trello ou GitHub Projects para o board. A coordenadora sinalizou que classificaria "quase tudo" como prioridade máxima na hora de priorizar.

Assinale as verdadeiras:

- [ ] 1ª (2): O item (II) já está pronto como está e não precisa de critérios de aceitação, pois seu benefício ("recuperar o acesso") é claro o suficiente para que a equipe saiba, sem ambiguidade, quando a story foi concluída.
- [ ] 2ª (4): O item (I) viola o critério *Small* do INVEST e, por reunir quatro áreas administrativas distintas, comporta-se mais como um Epic do que como uma user story, o que sugere quebrá-lo em stories menores antes de estimá-lo.
- [ ] 3ª (8): Diante de a coordenadora querer classificar "quase tudo" como prioridade máxima, a saída correta em MoSCoW é registrar a maior parte dos itens como *Must Have*, já que *Must* é justamente a categoria que acomoda tudo aquilo que o cliente considera importante.
- [ ] 4ª (16): Para o item (II), um critério de aceitação adequado em Gherkin seria: "Dado que a voluntária solicitou a redefinição, Quando ela abre o link após o prazo de validade, Então vê uma mensagem de erro e pode solicitar um novo link", um cenário de caminho triste tão legítimo quanto os de sucesso.
- [ ] 5ª (32): Marcar o item (I) como *Won't Have (now)* significaria descartá-lo definitivamente do produto, razão pela qual essa classificação deve ser evitada para funcionalidades administrativas que a ONG claramente vai precisar.
- [ ] 6ª (64): O item (III), embora bem formado quanto ao formato, falha no critério *Valuable* do INVEST: seu benefício está descrito da perspectiva técnica da equipe, não de um resultado percebido por quem usa o sistema, o que costuma indicar uma tarefa a ser absorvida por outra story, e não um item de backlog de produto por si só.
- [ ] 7ª (128): O item (III) é uma boa user story porque segue corretamente o formato "Como [persona], quero [ação], para [benefício]", e o formato correto é o que garante que a story entregue valor ao usuário.

Resposta (soma): _____

Questão 6

A equipe está no fim do Sprint 1 do **PetFood**. A funcionalidade de busca de produtos está pronta e passa em todos os critérios de aceitação de funcionalidade que a equipe escreveu. A busca foi implementada por dois integrantes que preferiram programar juntos no mesmo computador. Ao testar com a base real do pet shop, com milhares de produtos, notaram que a busca leva cerca de 8 segundos para retornar. O pet shop fica num bairro onde a internet costuma oscilar à tarde. Além disso, um integrante comentou que o código da busca "funciona, mas está tão embolado que, se precisarem mudar a regra de ordenação depois, vai ser um pesadelo". A equipe vai definir seu Definition of Done antes do Sprint 2.

Assinale as verdadeiras:

- ☐ 1ª (2): A melhor forma de garantir qualidade neste caso seria adiar qualquer preocupação com desempenho e manutenibilidade até uma fase final de verificação, quando todas as funcionalidades estiverem prontas e puderem ser avaliadas de uma só vez.
- ☐ 2ª (4): Embora a busca provavelmente responda rápido durante os testes da equipe, feitos com poucos produtos, os 8 segundos sob a base real mostram que ela cumpre a funcionalidade mas falha em desempenho, dois atributos distintos, e passar num não garante o outro.
- ☐ 3ª (8): Uma vez que a busca já passa em todos os critérios de aceitação escritos pela equipe, considerá-la "pronta" é a decisão correta; ajustes de desempenho e organização do código, se necessários, são melhorias que pertencem a um momento posterior à conclusão da funcionalidade.
- ☐ 4ª (16): O código "embolado" que dificultaria mudar a ordenação é um problema de manutenibilidade; e ainda que o usuário final nunca perceba a organização interna do código, isso não torna o problema menos real para a equipe, que arcará com o custo na evolução do sistema.
- ☐ 5ª (32): Como a lentidão de 8 segundos pode ser causada pela oscilação da internet do bairro à tarde, trata-se de um problema de infraestrutura externa, e não de um atributo de qualidade do sistema que a equipe precise endereçar.
- ☐ 6ª (64): Incluir no Definition of Done um critério como "toda funcionalidade nova deve ter o tempo de resposta verificado sob carga realista" operacionaliza qualidade de forma contínua, e o fato de o DoD anterior da equipe não conter nenhum critério de desempenho é justamente o que permitiu que esse problema só aparecesse agora, perto do fim do Sprint.

Resposta (soma): _____

Questão 7

Na aula de escopo assistido por IA, a equipe que atende a ONG **Raízes Urbanas** seguiu o roteiro proposto: primeiro, cada integrante escreveu sozinho, sem abrir a IA, o que entendia ser o escopo essencial do caso de treino; só depois compararam entre si e recorreram ao assistente. Um dos integrantes chegou atrasado e teve cinco minutos a menos para escrever sua versão. As versões individuais divergiam bastante entre si. Em seguida, deram à IA o papel de analista cético e pediram que ela apontasse personas esquecidas e o "problema por trás do problema". A sala estava com o ar-condicionado quebrado naquele dia. O escopo do MVP do projeto real já fora aprovado pela coordenadora em sprint anterior.

Assinale as verdadeiras:

- ☐ 1ª (2): Uma vez que a IA é boa em revelar personas esquecidas, faz sentido deixar que ela própria decida quais itens entram na fronteira do MVP, contanto que a equipe valide a lista no final.

- [ ] 2ª (4): Manter a divergência individual antes de recorrer à IA foi o que preservou a informação mais útil do exercício: o desacordo entre as visões, que some quando todos partem de uma mesma resposta pronta apresentada pela ferramenta.
- [ ] 3ª (8): O roteiro teria sido mais eficiente se a equipe tivesse aberto a IA primeiro, deixado que ela propusesse uma visão de escopo, e só depois pedido que cada integrante divergisse a partir dela, partindo de uma base estruturada em vez de versões cruas e conflitantes.
- [ ] 4ª (16): Dar à IA o papel de analista cético que questiona antes de gerar, em vez de pedir que ela defina o escopo, protege a equipe contra o viés de confirmação.
- [ ] 5ª (32): Ainda que a IA cética tenha feito boas perguntas, isso não equivale a ter elicitado a necessidade real: as lacunas que ela expõe entram no projeto como perguntas a levar à coordenadora, não como requisitos que a equipe já pode dar por levantados.
- [ ] 6ª (64): O uso adequado da IA sobre o escopo já aprovado seria pedir que ela reescrevesse o Canvas de Visão do zero com olhar cético, aproveitando que costuma produzir uma versão mais completa que a montada na correria da Semana 2.
- [ ] 7ª (128): Se a equipe não consegue explicar, sem o assistente, por que cada história está no backlog e que valor entrega, então, pela política da disciplina, essas histórias ainda não são propriamente da equipe.

Resposta (soma): _____

Questão 8

No **PetFood**, o frontend em React exibe a lista de produtos disponíveis; o backend em Express expõe a rota `GET /api/produtos`; os dados ficam num banco PostgreSQL. A equipe hesitou entre chamar a rota de `/api/produtos` ou `/api/products` e acabou padronizando em português. Um integrante colocou, dentro do controller dessa rota, tanto a montagem da resposta HTTP quanto a regra de negócio "só retornar produtos com estoque maior que zero" quanto o comando SQL que acessa o banco. Esse integrante é o único da equipe que já tinha mexido com SQL antes. Durante a Sprint Review, a coordenadora pediu que, no futuro, produtos esgotados também aparecessem, porém marcados como indisponíveis.

Assinale as verdadeiras:

- [ ] 1ª (2): Se o time de frontend precisar exibir os produtos esgotados como indisponíveis, terá de esperar que o backend altere o comando SQL interno, pois é o formato do SQL que o contrato da rota `GET /api/produtos` expõe ao cliente.
- [ ] 2ª (4): Se a equipe extraísse o comando SQL para uma camada de repositório, o controller passaria a depender da interface desse repositório em vez do SQL concreto; assim, trocar PostgreSQL por outro banco tenderia a afetar o repositório, e não o controller nem a regra de negócio.
- [ ] 3ª (8): Como o React roda no navegador do usuário e o Express num servidor, os dois estão em processos separados; logo, a regra "estoque maior que zero" poderia

ser implementada tanto no React quanto no Express sem diferença arquitetural, já que ambos são código JavaScript da mesma equipe.

- [ ] 4ª (16): Para atender o novo pedido da coordenadora (mostrar esgotados como indisponíveis), a equipe precisará alterar a regra de filtragem que hoje vive no controller; se essa regra estivesse isolada numa camada de serviço, a rota `GET /api/produtos` e o código de resposta HTTP poderiam permanecer intocados.
- [ ] 5ª (32): Com a regra de estoque, o SQL e a resposta HTTP no mesmo controller, um colega que quisesse alterar apenas o formato do JSON de resposta precisaria abrir e entender também a regra de negócio e a consulta ao banco, mesmo sem ter intenção de mexer em nenhuma das duas.

Resposta (soma): _____

Questão 9

Na equipe da ONG **Raízes Urbanas**, dois integrantes trabalham no mesmo arquivo `voluntarioController.js`. Os dois combinaram de programar em horários alternados para não se atrapalhar, um de manhã e outro à noite. Esse arquivo, com cerca de 250 linhas, contém a validação de CPF, o envio do e-mail de boas-vindas, o acesso ao banco e a montagem da resposta HTTP, tudo em uma função chamada `handle()`. Ao tentar fazer merge das duas contribuições no fim da semana, o GitHub acusou vários conflitos no mesmo trecho. Um dos integrantes usa tema escuro no editor e o outro tema claro. Um deles precisou mudar só o texto do e-mail de boas-vindas e, para isso, teve de abrir e reler a função inteira.

Assinale as verdadeiras:

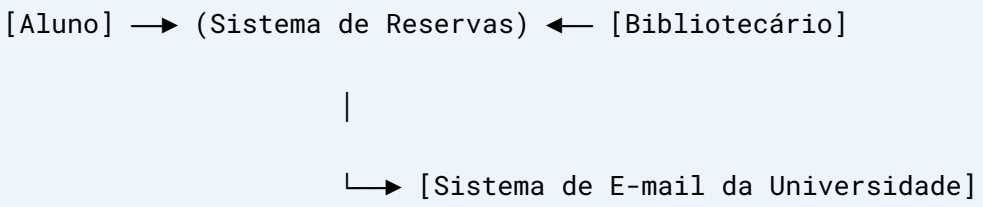
- [ ] 1ª (2): Renomear `handle()` para `handleCadastroVoluntario()` tornaria a intenção do código mais evidente para quem o lê pela primeira vez e, por deixar o propósito explícito no nome, seria suficiente para resolver a dificuldade que a equipe tem de alterar esse arquivo sem efeitos colaterais.
- [ ] 2ª (4): Enquanto a montagem da resposta HTTP, a validação de CPF e a consulta ao banco compartilharem o corpo de `handle()`, uma correção no texto do e-mail e uma correção na regra de validação, feitas por pessoas diferentes na mesma semana, tendem a recair sobre linhas vizinhas do mesmo bloco, e é aí que o merge acusa conflito.
- [ ] 3ª (8): Quebrar as 250 linhas em dois arquivos de cerca de 125, deixando em cada um um pedaço da validação, um pedaço do e-mail e um pedaço do acesso ao banco misturados, reduziria o número de linhas por arquivo, mas dois desenvolvedores mexendo em responsabilidades distintas ainda poderiam colidir, agora espalhados por dois arquivos em vez de um.
- [ ] 4ª (16): Se os dois integrantes passassem a trabalhar no mesmo turno e a integrar várias vezes ao dia, os conflitos de merge neste arquivo deixariam de ocorrer, pois estariam sempre partindo da versão mais recente um do outro antes de alterar qualquer parte de `handle()`.

- [ ] 5ª (32): Extrair a validação de CPF para uma função própria permitiria escrever um teste que verifica se um CPF malformado é rejeitado, sem que esse teste dispare o envio de e-mail nem precise de uma conexão com o banco para rodar.
- [ ] 6ª (64): Se o envio do e-mail passasse para uma função `enviarEmailBoasVindas(dados)` chamada por `handle()`, uma futura troca do serviço de e-mail, do modo atual para um provedor com API diferente, poderia ser feita alterando apenas essa função, sem reabrir a lógica de validação nem a de resposta.

Resposta (soma): _____

Questão 10

Equipe de 4 pessoas, 4 meses, para construir um sistema de reservas de uma biblioteca universitária, hoje um monólito Client/Server (React + Express em camadas + um banco PostgreSQL). Na Sprint 1 Review, a equipe levou este diagrama:



E registrou no repositório:

> ***ADR-003 — Banco único*** *Contexto: equipe de 4, prazo de 4 meses, sem experiência prévia em operar serviços distribuídos. Decisão: manter um único banco PostgreSQL para todo o sistema. Consequências: simplifica deploy e desenvolvimento agora; se reservas e catálogo crescerem muito, pode ser necessário revisitar.*

Dois integrantes queriam microsserviços porque viram num vídeo que "é o que as big techs usam". O bibliotecário perguntou se haveria app de celular, o que não está no escopo.

Assinale as verdadeiras:

- [ ] 1ª (2): Diante da insistência nos microsserviços, um integrante propõe encerrar a discussão declarando que o monólito atual é uma escolha "sem trade-offs" para este projeto, já que não se enxerga nenhuma desvantagem nele no prazo de 4 meses, e que isso prova objetivamente que é a decisão certa.
- [ ] 2ª (4): Dois integrantes defendem separar reservas e catálogo em bancos distintos agora, argumentando que o *Bounded Context* prega que cada serviço seja dono do seu dado e que, portanto, o banco único já é uma violação arquitetural a corrigir antes que o acoplamento cresça. Seguir esse argumento seria um erro: o Bounded Context governa *como* delimitar serviços que a equipe decidiu ter, e não é razão, por si, para introduzir a separação de serviços num sistema cujo contexto (4 pessoas, 4 meses) não a justifica.

- [ ] 3ª (8): Como o bibliotecário, na Review, não conseguiu opinar sobre decisões técnicas e ainda perguntou sobre um app de celular fora do escopo, um integrante conclui que o diagrama de Contexto foi inadequado para aquela audiência e que teria sido melhor levar o diagrama de Contêiner, mais detalhado, para dar a ele base técnica de avaliação.
- [ ] 4ª (16): Um integrante afirma que registrar o ADR-003 foi esforço perdido: como a arquitetura provavelmente vai mudar ao longo do semestre, documentar agora uma decisão que pode ser revista é trabalho que será jogado fora assim que a separação de bancos acontecer. Esse argumento não se sustenta: quando a separação ocorrer, um novo ADR registrará a mudança, e o ADR-003 seguirá explicando em que contexto o banco único fez sentido, de modo que o registro atual não é descartado, e sim torna-se o histórico que evita rediscutir a escolha original.
- [ ] 5ª (32): Para a próxima Review, a equipe quer mostrar ao bibliotecário como React, Express e banco se distribuem e se comunicam. Um integrante propõe apenas anotar as tecnologias sobre as caixas do diagrama atual, mantendo os mesmos elementos, e conclui que isso já o converte num diagrama de Contêiner do C4, pronto para essa finalidade. A proposta é adequada, pois é a indicação das tecnologias sobre os elementos existentes que caracteriza a passagem do Nível de Contexto para o Nível de Contêiner.
- [ ] 6ª (64): Alguém sugere apagar o campo *Consequências* do ADR-003 e deixá-lo só com Contexto e Decisão, sob o argumento de que, como a decisão de banco único claramente foi acertada para o momento, registrar uma possível revisão futura só abre margem para questionamentos desnecessários. Essa edição enfraqueceria o ADR: é justamente o registro do custo e da condição de revisão que impede a equipe futura de tratar a escolha como definitiva e de refazer a análise do zero quando o contexto mudar.
- [ ] 7ª (128): Os defensores dos microsserviços argumentam que, como o próprio ADR-003 admite que a separação pode ser necessária se reservas e catálogo crescerem, o mais eficiente seria já nascer com os serviços separados, evitando o custo de migrar um monólito depois, afinal, refatorar de monólito para microsserviços é sabidamente caro. Esse raciocínio deve ser recusado no contexto dado: antecipar a separação impõe hoje, a uma equipe sem experiência distribuída e com 4 meses, um custo operacional certo para evitar um custo futuro que é apenas possível e condicional ao crescimento.

Resposta (soma): _____

Questão 11 - bônus

Na equipe da ONG **Raízes Urbanas**, quatro pessoas trabalham no mesmo repositório. Durante a Sprint, combinaram que cada funcionalidade seria desenvolvida em um branch próprio e integrada à `main` via Pull Request revisado por outro colega. Dois integrantes preferem fazer commits no fim do dia; os outros dois, a cada poucas horas. Numa dada semana: a integrante A abriu um branch para o cadastro de doações e passou seis dias sem integrá-lo, acumulando muitas mudanças; o integrante B abriu um PR pequeno e pediu revisão, mas ninguém revisou por três dias; e a dupla C/D, trabalhando no mesmo

arquivo em branches diferentes, adiou a integração para o último dia. A `main` está protegida contra push direto. Ao final, houve conflitos grandes e demorados de resolver.

Assinale as verdadeiras:

- [ ] 1ª (2): Como dois integrantes commitam só no fim do dia e os outros dois a cada poucas horas, e sabendo que integrações mais frequentes tendem a produzir conflitos menores, a conclusão coerente é que padronizar todos para commitar apenas uma vez ao dia alinharia a cadência da equipe e, com uma única janela diária de integração, concentraria e reduziria os conflitos.
- [ ] 2ª (4): Já que o conflito concreto surgiu entre C e D por editarem o mesmo arquivo em paralelo, e como o Git resolve automaticamente merges quando as pessoas trabalham em arquivos diferentes, a lição consistente com o funcionamento do Git é organizar a divisão de tarefas de modo que dois integrantes nunca fiquem responsáveis por um mesmo arquivo na mesma Sprint, eliminando na origem a possibilidade de conflito.
- [ ] 3ª (8): Como o objetivo de um PR é dar a outro colega a chance de revisar antes da integração, e o PR de B ficou três dias parado sem revisor, a equipe deveria concluir que, quando não há disponibilidade de revisão em tempo hábil, o mais alinhado ao propósito da revisão é autorizar o próprio autor a aprovar e integrar o seu PR, para que o fluxo não trave, a revisão que não acontece a tempo perde a função.
- [ ] 4ª (16): Considerando que a base da `main` evolui enquanto um branch permanece isolado, o branch de seis dias de A e a integração de última hora de C/D compartilham a mesma causa de fundo: ambos deixaram a distância entre o seu trabalho e a `main` crescer sem reconciliação, de modo que o esforço de merge não foi criado no dia do conflito, mas acumulado silenciosamente nos dias anteriores em que a integração foi adiada.
- [ ] 5ª (32): Uma vez que a `main` está protegida contra push direto, e que essa proteção obriga toda mudança a passar por um branch e um PR antes de chegar à linha principal, é coerente concluir que foi essa exigência de intermediação que represou as mudanças em branches separados por vários dias — logo, afrouxar a proteção para permitir push direto em casos urgentes reduziria o represamento e, com ele, os conflitos.

Resposta (soma): _____

CIN0136 — Desenvolvimento de Software
Bacharelado em Sistemas de Informação ▪ CIn - UFPE
Profs. Kiev Santos da Gama & Vinicius Cardoso Garcia

✏ Exercício Escolar 1C
13/10/2026

Instruções: Leia as questões com atenção e cuidado, e responda com atenção ao limite de caracteres, quando indicado.

- Esta avaliação é **individual**, **sem consulta**, e vale até **10 pontos** no total. A última questão (Q11) é **bônus** e soma por cima (sua nota pode passar de 10,0).
- A duração total é de **até 2 horas**; organize seu tempo..
- Consulta a qualquer material: **não permitida**.
- Uso de IA ou dispositivos eletrônicos: **não permitido**.
- Leia cada questão com atenção antes de responder. Gerencie bem seu tempo.
- Respostas vagas ou sem justificativa serão consideradas incompletas.
- Dúvidas podem ser expostas, **publicamente**, durante **os primeiros 30 minutos** da realização do Exercício Escolar. **Entender o enunciado** faz parte da avaliação.
- Não é permitido ir ao sanitário durante a realização do exercício, **vá antes**.

Como responder (modelo somatório).

- Cada questão tem um cenário e de 5 a 7 alternativas. Cada alternativa vale um número em potência de 2, conforme sua **posição**: a 1ª vale **2**, a 2ª **4**, a 3ª **8**, a 4ª **16**, a 5ª **32**, a 6ª **64**, a 7ª **128**.
- Marque as alternativas que julgar **verdadeiras** e escreva, no campo de resposta, a **soma** dos valores marcados.
- Correção:
  - Marcou **qualquer alternativa falsa** → a questão vale **0,0** (zera inteira).
  - Marcou **só verdadeiras, mas não todas** → nota proporcional: (nº de corretas marcadas ÷ nº total de corretas) × 1,0.
  - Marcou **todas as corretas e nenhuma falsa** → **1,0**.

Questão 1

A equipe que atende a ONG **Raízes Urbanas** recebeu, de um voluntário que é estudante de programação, um script em Python que gera automaticamente os relatórios mensais de doações. O script funciona: rodado na máquina do voluntário, produz o relatório correto em segundos. O voluntário mencionou que levou só duas tardes para escrevê-lo. A coordenadora da ONG gostou do resultado e pediu que a equipe incorporasse essa geração de relatórios ao sistema que estão construindo, que usa Node.js. Ao abrir o script, a equipe encontra um único arquivo de 300 linhas, sem nenhum teste, sem comentários, com todas as regras de cálculo e o acesso ao banco misturados na mesma função. O voluntário se formou e não está mais disponível para explicar o código.

Assinale as verdadeiras:

- ☐ 1ª (2): Incorporar o script no estado atual permitiria entregar valor à coordenadora rapidamente; como o comportamento já foi validado na prática, o esforço de

reorganizá-lo antes da incorporação seria uma otimização prematura neste momento do projeto.

- ☐ 2ª (4): Que o script "funcione" e produza o relatório correto é insuficiente para classificá-lo como software de qualidade, porque atributos como manutenibilidade e testabilidade não estão presentes, e são justamente eles que permitirão a evolução do relatório no futuro.
- ☐ 3ª (8): Concentrar as regras de cálculo e o acesso ao banco em uma única função de 300 linhas favorece a legibilidade, por manter todo o fluxo do relatório visível num só lugar e evitar a dispersão da separação em arquivos.
- ☐ 4ª (16): Como a saída do relatório já foi conferida pela coordenadora e está correta, a incorporação pode prosseguir sem testes automatizados por ora, ficando a cobertura de testes como uma melhoria de qualidade desejável, mas não relacionada ao risco de a lógica quebrar em alterações futuras.
- ☐ 5ª (32): A dificuldade que a equipe terá para modificar o script, agravada pela saída do voluntário, é um desafio de natureza tanto técnica (código sem estrutura) quanto organizacional (perda de conhecimento e ausência de documentação).

Resposta (soma): _____

Questão 2

Numa aula de revisão, o professor propõe o seguinte caso para discussão. Uma equipe precisa construir um aplicativo de cadastro de beneficiários para uma ONG. A equipe tem cinco integrantes, um dos quais nunca havia trabalhado com banco de dados antes. Duas abordagens são colocadas na mesa. Na **Abordagem 1**, a equipe entrega na primeira semana apenas o formulário de nome; na segunda, acrescenta o formulário de endereço; na terceira, o histórico de atendimentos (cada entrega adiciona uma parte nova e completa do sistema). Na **Abordagem 2**, a equipe entrega na primeira semana uma versão do sistema inteiro com os três campos, porém simples, feia e sem validação; nas semanas seguintes, refina essa mesma versão, adicionando validação, melhorando a interface e ajustando com base no feedback. A coordenadora da ONG só poderá dar feedback a partir da terceira semana, por estar viajando.

Assinale as verdadeiras:

- ☐ 1ª (2): A Abordagem 1 descreve um modelo incremental, pois o produto cresce por adição de partes novas e completas a cada entrega, enquanto a Abordagem 2 descreve um modelo iterativo, em que a mesma versão é refinada sucessivamente.
- ☐ 2ª (4): O modelo cascata foi um erro histórico da engenharia de software e não possui nenhum contexto de aplicação legítimo nos dias de hoje, tendo sido inteiramente substituído pelos modelos incremental e iterativo.
- ☐ 3ª (8): A Abordagem 1 incorpora o feedback do usuário de forma mais natural que a Abordagem 2, pois, ao entregar uma parte nova e completa a cada semana, permite que cada incremento seja revisto e retrabalhado sobre si mesmo antes do próximo.

- [ ] 4ª (16): Ambas as abordagens entregam software ao longo do tempo em vez de tudo no final, o que as diferencia do modelo cascata clássico, no qual a validação do produto pelo usuário tende a ocorrer tardiamente.
- [ ] 5ª (32): A Abordagem 2 é preferível à Abordagem 1 em qualquer projeto, porque refinar uma versão existente é sempre menos arriscado do que adicionar módulos novos, que podem quebrar o que já funciona.
- [ ] 6ª (64): No modelo iterativo, cada ciclo assume explicitamente que a equipe vai aprender algo e que o produto vai mudar, o que torna o feedback (quando disponível) uma entrada planejada do processo, e não uma interrupção dele.

Resposta (soma): _____

Questão 3

A equipe que atende a ONG **Raízes Urbanas** está encerrando a terceira Sprint. Nesta Sprint, entregaram o módulo de cadastro de voluntários e corrigiram três bugs reportados pela coordenadora na Sprint anterior. Durante a semana houve um desentendimento entre dois integrantes sobre qual editor de código a equipe deveria padronizar, o que gerou certo mal-estar. A equipe adotou Scrum com Sprints de duas semanas e usa um board no GitHub Projects para visualizar o fluxo de trabalho. A coordenadora confirmou presença no evento de fechamento da Sprint. O integrante responsável por guardar e priorizar o backlog percebeu que o board acumulou muitos itens simultaneamente na coluna "Em progresso".

Assinale as verdadeiras:

- [ ] 1ª (2): O evento de Sprint Review, ao final da Sprint, serve para demonstrar o incremento à coordenadora e coletar seu feedback sobre o que foi construído, enquanto a reflexão da equipe sobre o próprio processo de trabalho pertence à Retrospectiva.
- [ ] 2ª (4): Como a coordenadora estará presente no fechamento da Sprint, a equipe deve aproveitar esse momento para, além de apresentar o incremento, resolver ali o desentendimento sobre o editor de código, usando a presença de uma figura externa para mediar o conflito interno.
- [ ] 3ª (8): Ao aplicar um limite de WIP na coluna "Em progresso", a equipe passa a controlar o fluxo de forma tão eficaz que o Sprint Planning se torna dispensável, já que limitar itens simultâneos cumpre o mesmo papel de definir o que a equipe se compromete a entregar no ciclo.
- [ ] 4ª (16): O acúmulo de muitos itens simultâneos em "Em progresso" contraria a lógica de limitar o trabalho em progresso (WIP); reduzir os itens em andamento tende a diminuir o cycle time, acelerando a entrega efetiva de cada item.
- [ ] 5ª (32): O Daily Standup tem como propósito sincronizar a equipe em torno do que foi feito, do que será feito e dos impedimentos, e não substitui a Sprint Review, cujo foco é a inspeção do incremento com o stakeholder.
- [ ] 6ª (64): O papel de guardar e priorizar o backlog, exercido pelo integrante mencionado, corresponde ao do Product Owner, responsável por representar os interesses do stakeholder na ordenação do que será construído.

Resposta (soma): _____

Questão 4

A equipe está levantando requisitos do **PetFood**, o sistema de gestão de produtos para pet shop. O dono do pet shop mencionou que pretende, no futuro, abrir uma segunda loja em outro bairro. Durante uma visita, a equipe observou a atendente trabalhando: ela consultava os preços num sistema antigo, mas mantinha ao lado um caderno onde anotava quais clientes costumavam comprar ração de marcas específicas, informação que o sistema atual não guardava. A atendente comentou que o caderno era um presente de aniversário. Em conversa posterior, o dono afirmou que o sistema "precisa ser rápido" e que "qualquer funcionário novo tem que conseguir usar sem treinamento". A equipe também registrou que o sistema deve permitir cadastrar produtos com nome, preço e categoria.

Assinale as verdadeiras:

- ☐ 1ª (2): "Cadastrar produtos com nome, preço e categoria" e "usar o sistema sem treinamento" são requisitos de naturezas distintas: o primeiro descreve uma função verificável por presença ou ausência, o segundo é um atributo de qualidade cuja verificação depende de definir o que conta como "sem treinamento".
- ☐ 2ª (4): A afirmação de que o sistema "precisa ser rápido", tal como o dono a expressou, já pode ser levada diretamente para a fase de testes e verificada de forma objetiva, sem necessidade de trabalho adicional de refinamento com o cliente.
- ☐ 3ª (8): Por "usar sem treinamento" ser um requisito não funcional de usabilidade, sua verificação é necessariamente subjetiva e deve ficar a cargo da impressão de cada funcionário novo, não cabendo à equipe traduzi-lo em algum critério observável.
- ☐ 4ª (16): A descoberta do caderno pela observação da atendente em seu ambiente real cumpre um papel que a entrevista com o dono dificilmente cumpriria sozinha: expor uma necessidade que ninguém verbaliza porque, para quem trabalha ali, ela já é parte invisível da rotina.
- ☐ 5ª (32): O caderno indica um requisito que o sistema atual não atende, e o fato de ele ter surgido da observação, e não da fala do dono, ilustra por que restringir a elicitação a perguntar "o que você quer?" tende a produzir um sistema que ignora parte do trabalho real.

Resposta (soma): _____

Questão 5

A equipe que atende a ONG **Raízes Urbanas** está refinando o backlog antes da sessão com a coordenadora. A reunião foi remarcada duas vezes por causa de um feriado municipal. Entre os itens levantados, aparecem estes três: **(I)** "Como coordenadora, quero gerenciar voluntários, doações, eventos e relatórios para administrar a ONG"; **(II)** "Como

voluntária, quero redefinir minha senha para recuperar o acesso à minha conta quando esquecê-la"; **(III)** "Como desenvolvedor, quero migrar o banco para PostgreSQL para melhorar a performance". A equipe discutiu por dez minutos se usaria Trello ou GitHub Projects para o board. A coordenadora sinalizou que classificaria "quase tudo" como prioridade máxima na hora de priorizar.

Assinale as verdadeiras:

- [ ] 1ª (2): O item (I) viola o critério *Small* do INVEST e, por reunir quatro áreas administrativas distintas, comporta-se mais como um Epic do que como uma user story, o que sugere quebrá-lo em stories menores antes de estimá-lo.
- [ ] 2ª (4): O item (III) é uma boa user story porque segue corretamente o formato "Como [persona], quero [ação], para [benefício]", e o formato correto é o que garante que a story entregue valor ao usuário.
- [ ] 3ª (8): Para o item (II), um critério de aceitação adequado em Gherkin seria: "Dado que a voluntária solicitou a redefinição, Quando ela abre o link após o prazo de validade, Então vê uma mensagem de erro e pode solicitar um novo link", um cenário de caminho triste tão legítimo quanto os de sucesso.
- [ ] 4ª (16): Diante de a coordenadora querer classificar "quase tudo" como prioridade máxima, a saída correta em MoSCoW é registrar a maior parte dos itens como *Must Have*, já que *Must* é justamente a categoria que acomoda tudo aquilo que o cliente considera importante.
- [ ] 5ª (32): Para o item (II) ser considerado *Testable* no INVEST, basta especificar em Gherkin o cenário de sucesso da redefinição de senha; os cenários de falha, por tratarem de exceções, são refinamentos opcionais que não afetam se a story pode ou não ser verificada.
- [ ] 6ª (64): O item (III), embora bem formado quanto ao formato, falha no critério *Valuable* do INVEST: seu benefício está descrito da perspectiva técnica da equipe, não de um resultado percebido por quem usa o sistema, o que costuma indicar uma tarefa a ser absorvida por outra story, e não um item de backlog de produto por si só.
- [ ] 7ª (128): Marcar o item (I) como *Won't Have (now)* significaria descartá-lo definitivamente do produto, razão pela qual essa classificação deve ser evitada para funcionalidades administrativas que a ONG claramente vai precisar.

Resposta (soma): _____

Questão 6

A equipe está no fim do Sprint 1 do **PetFood**. A funcionalidade de busca de produtos está pronta e passa em todos os critérios de aceitação de funcionalidade que a equipe escreveu. A busca foi implementada por dois integrantes que preferiram programar juntos no mesmo computador. Ao testar com a base real do pet shop, com milhares de produtos, notaram que a busca leva cerca de 8 segundos para retornar. O pet shop fica num bairro onde a internet costuma oscilar à tarde. Além disso, um integrante comentou que o código da busca "funciona, mas está tão embolado que, se precisarem mudar a regra de

ordenação depois, vai ser um pesadelo". A equipe vai definir seu Definition of Done antes do Sprint 2.

Assinale as verdadeiras:

- ☐ 1ª (2): Embora a busca provavelmente responda rápido durante os testes da equipe, feitos com poucos produtos, os 8 segundos sob a base real mostram que ela cumpre a funcionalidade mas falha em desempenho, dois atributos distintos, e passar num não garante o outro.
- ☐ 2ª (4): A melhor forma de garantir qualidade neste caso seria adiar qualquer preocupação com desempenho e manutenibilidade até uma fase final de verificação, quando todas as funcionalidades estiverem prontas e puderem ser avaliadas de uma só vez.
- ☐ 3ª (8): Os 8 segundos são de fato lentidão do sistema sob carga, mas, como nenhum critério de aceitação da equipe fixou um tempo máximo de resposta, o desempenho aqui é uma preferência subjetiva do cliente, e não um atributo de qualidade que se possa avaliar objetivamente.
- ☐ 4ª (16): O código "embolado" que dificultaria mudar a ordenação é um problema de manutenibilidade; e ainda que o usuário final nunca perceba a organização interna do código, isso não torna o problema menos real para a equipe, que arcará com o custo na evolução do sistema.
- ☐ 5ª (32): Incluir no Definition of Done um critério como "toda funcionalidade nova deve ter o tempo de resposta verificado sob carga realista" operacionaliza qualidade de forma contínua, e o fato de o DoD anterior da equipe não conter nenhum critério de desempenho é justamente o que permitiu que esse problema só aparecesse agora, perto do fim do Sprint.
- ☐ 6ª (64): Uma vez que a busca já passa em todos os critérios de aceitação escritos pela equipe, considerá-la "pronta" é a decisão correta; ajustes de desempenho e organização do código, se necessários, são melhorias que pertencem a um momento posterior à conclusão da funcionalidade.

Resposta (soma): _____

Questão 7

Na aula de escopo assistido por IA, a equipe que atende a ONG **Raízes Urbanas** seguiu o roteiro proposto: primeiro, cada integrante escreveu sozinho, sem abrir a IA, o que entendia ser o escopo essencial do caso de treino; só depois compararam entre si e recorreram ao assistente. Um dos integrantes chegou atrasado e teve cinco minutos a menos para escrever sua versão. As versões individuais divergiam bastante entre si. Em seguida, deram à IA o papel de analista cético e pediram que ela apontasse personas esquecidas e o "problema por trás do problema". A sala estava com o ar-condicionado quebrado naquele dia. O escopo do MVP do projeto real já fora aprovado pela coordenadora em sprint anterior.

Assinale as verdadeiras:

- [ ] 1ª (2): Dar à IA o papel de analista cético que questiona antes de gerar, em vez de pedir que ela defina o escopo, protege a equipe contra o viés de confirmação.
- [ ] 2ª (4): O roteiro teria sido mais eficiente se a equipe tivesse aberto a IA primeiro, deixado que ela propusesse uma visão de escopo, e só depois pedido que cada integrante divergisse a partir dela, partindo de uma base estruturada em vez de versões cruas e conflitantes.
- [ ] 3ª (8): Se a equipe não consegue explicar, sem o assistente, por que cada história está no backlog e que valor entrega, então, pela política da disciplina, essas histórias ainda não são propriamente da equipe.
- [ ] 4ª (16): Ao usar a IA como advogado do diabo da fronteira do MVP, a regra é: se a equipe não consegue rebater o contra-argumento da IA contra um item, deve ceder e ajustar a fronteira conforme a IA sugeriu, pois isso significa que o argumento da IA era mais forte.
- [ ] 5ª (32): Ainda que a IA cética tenha feito boas perguntas, isso não equivale a ter elicitado a necessidade real: as lacunas que ela expõe entram no projeto como perguntas a levar à coordenadora, não como requisitos que a equipe já pode dar por levantados.
- [ ] 6ª (64): O uso adequado da IA sobre o escopo já aprovado seria pedir que ela reescrevesse o Canvas de Visão do zero com olhar cético, aproveitando que costuma produzir uma versão mais completa que a montada na correria da Semana 2.
- [ ] 7ª (128): Manter a divergência individual antes de recorrer à IA foi o que preservou a informação mais útil do exercício: o desacordo entre as visões, que some quando todos partem de uma mesma resposta pronta apresentada pela ferramenta.

Resposta (soma): _____

Questão 8

No **PetFood**, o frontend em React exibe a lista de produtos disponíveis; o backend em Express expõe a rota `GET /api/produtos`; os dados ficam num banco PostgreSQL. A equipe hesitou entre chamar a rota de `/api/produtos` ou `/api/products` e acabou padronizando em português. Um integrante colocou, dentro do controller dessa rota, tanto a montagem da resposta HTTP quanto a regra de negócio "só retornar produtos com estoque maior que zero" quanto o comando SQL que acessa o banco. Esse integrante é o único da equipe que já tinha mexido com SQL antes. Durante a Sprint Review, a coordenadora pediu que, no futuro, produtos esgotados também aparecessem, porém marcados como indisponíveis.

Assinale as verdadeiras:

- [ ] 1ª (2): Para atender o novo pedido da coordenadora (mostrar esgotados como indisponíveis), a equipe precisará alterar a regra de filtragem que hoje vive no controller; se essa regra estivesse isolada numa camada de serviço, a rota `GET /api/produtos` e o código de resposta HTTP poderiam permanecer intocados.
- [ ] 2ª (4): Como a rota `GET /api/produtos` é um contrato entre frontend e backend, qualquer alteração na regra interna de estoque, inclusive passar a incluir esgotados

como indisponíveis, obriga a alterar também o contrato da rota, pois o cliente precisa ser avisado de que a lógica por trás mudou.

- [ ] 3ª (8): Como o React roda no navegador do usuário e o Express num servidor, os dois estão em processos separados; logo, a regra "estoque maior que zero" poderia ser implementada tanto no React quanto no Express sem diferença arquitetural, já que ambos são código JavaScript da mesma equipe.
- [ ] 4ª (16): Com a regra de estoque, o SQL e a resposta HTTP no mesmo controller, um colega que quisesse alterar apenas o formato do JSON de resposta precisaria abrir e entender também a regra de negócio e a consulta ao banco, mesmo sem ter intenção de mexer em nenhuma das duas.
- [ ] 5ª (32): Se a equipe extraísse o comando SQL para uma camada de repositório, o controller passaria a depender da interface desse repositório em vez do SQL concreto; assim, trocar PostgreSQL por outro banco tenderia a afetar o repositório, e não o controller nem a regra de negócio.

Resposta (soma): _____

Questão 9

Na equipe da ONG **Raízes Urbanas**, dois integrantes trabalham no mesmo arquivo `voluntarioController.js`. Os dois combinaram de programar em horários alternados para não se atrapalhar, um de manhã e outro à noite. Esse arquivo, com cerca de 250 linhas, contém a validação de CPF, o envio do e-mail de boas-vindas, o acesso ao banco e a montagem da resposta HTTP — tudo em uma função chamada `handle()`. Ao tentar fazer merge das duas contribuições no fim da semana, o GitHub acusou vários conflitos no mesmo trecho. Um dos integrantes usa tema escuro no editor e o outro tema claro. Um deles precisou mudar só o texto do e-mail de boas-vindas e, para isso, teve de abrir e reler a função inteira.

Assinale as verdadeiras:

- [ ] 1ª (2): Enquanto a montagem da resposta HTTP, a validação de CPF e a consulta ao banco compartilharem o corpo de `handle()`, uma correção no texto do e-mail e uma correção na regra de validação, feitas por pessoas diferentes na mesma semana, tendem a recair sobre linhas vizinhas do mesmo bloco, e é aí que o merge acusa conflito.
- [ ] 2ª (4): Se os dois integrantes passassem a trabalhar no mesmo turno e a integrar várias vezes ao dia, os conflitos de merge neste arquivo deixariam de ocorrer, pois estariam sempre partindo da versão mais recente um do outro antes de alterar qualquer parte de `handle()`.
- [ ] 3ª (8): Extrair somente a validação de CPF para uma função separada já bastaria para eliminar o problema de manutenção do arquivo, pois, uma vez isolada a validação, as demais partes (envio de e-mail, acesso ao banco e montagem da resposta) deixariam de interferir umas nas outras dentro de `handle()`.
- [ ] 4ª (16): Quebrar as 250 linhas em dois arquivos de cerca de 125, deixando em cada um um pedaço da validação, um pedaço do e-mail e um pedaço do acesso ao banco misturados, reduziria o número de linhas por arquivo, mas dois

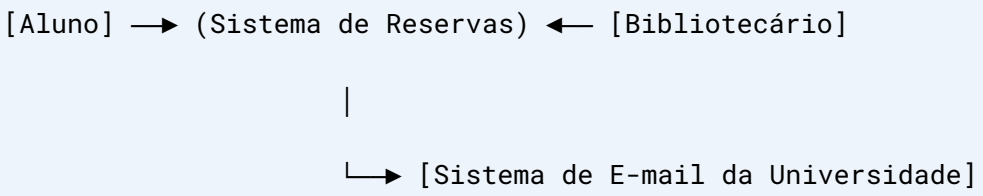
desenvolvedores mexendo em responsabilidades distintas ainda poderiam colidir, agora espalhados por dois arquivos em vez de um.

- ☐ 5ª (32): Extrair a validação de CPF para uma função própria permitiria escrever um teste que verifica se um CPF malformado é rejeitado, sem que esse teste dispare o envio de e-mail nem precise de uma conexão com o banco para rodar.
- ☐ 6ª (64): Se o envio do e-mail passasse para uma função `enviarEmailBoasVindas(dados)` chamada por `handle()`, uma futura troca do serviço de e-mail (do modo atual para um provedor com API diferente) poderia ser feita alterando apenas essa função, sem reabrir a lógica de validação nem a de resposta.

Resposta (soma): _____

Questão 10

Equipe de 4 pessoas, 4 meses, para construir um sistema de reservas de uma biblioteca universitária, hoje um monólito Client/Server (React + Express em camadas + um banco PostgreSQL). Na Sprint 1 Review, a equipe levou este diagrama:



E registrou no repositório:

> ***ADR-003 — Banco único*** *Contexto: equipe de 4, prazo de 4 meses, sem experiência prévia em operar serviços distribuídos. Decisão: manter um único banco PostgreSQL para todo o sistema. Consequências: simplifica deploy e desenvolvimento agora; se reservas e catálogo crescerem muito, pode ser necessário revisitar.*

Dois integrantes queriam microsserviços porque viram num vídeo que "é o que as big techs usam". O bibliotecário perguntou se haveria app de celular, o que não está no escopo.

Assinale as verdadeiras:

- ☐ 1ª (2): Dois integrantes defendem separar reservas e catálogo em bancos distintos agora, argumentando que o *Bounded Context* prega que cada serviço seja dono do seu dado e que, portanto, o banco único já é uma violação arquitetural a corrigir antes que o acoplamento cresça. Seguir esse argumento seria um erro: o Bounded Context governa *como* delimitar serviços que a equipe decidiu ter, e não é razão, por si, para introduzir a separação de serviços num sistema cujo contexto (4 pessoas, 4 meses) não a justifica.
- ☐ 2ª (4): Diante da insistência nos microsserviços, um integrante propõe encerrar a discussão declarando que o monólito atual é uma escolha "sem trade-offs" para

este projeto, já que não se enxerga nenhuma desvantagem nele no prazo de 4 meses, e que isso prova objetivamente que é a decisão certa.

- [ ] 3ª (8): Alguém sugere apagar o campo *Consequências* do ADR-003 e deixá-lo só com Contexto e Decisão, sob o argumento de que, como a decisão de banco único claramente foi acertada para o momento, registrar uma possível revisão futura só abre margem para questionamentos desnecessários. Essa edição enfraqueceria o ADR: é justamente o registro do custo e da condição de revisão que impede a equipe futura de tratar a escolha como definitiva e de refazer a análise do zero quando o contexto mudar.
- [ ] 4ª (16): Como o Diagrama de Contexto é o nível adequado para uma audiência não técnica, a equipe pode considerar que a aprovação do bibliotecário na Review valida que a arquitetura do sistema foi tecnicamente bem construída: se o único diagrama que ele precisa ver é o N1 e ele o aprovou, a corretude das decisões internas está chancelada por quem importa.
- [ ] 5ª (32): Para a próxima Review, a equipe quer mostrar ao bibliotecário como React, Express e banco se distribuem e se comunicam. Um integrante propõe apenas anotar as tecnologias sobre as caixas do diagrama atual, mantendo os mesmos elementos, e conclui que isso já o converte num diagrama de Contêiner do C4, pronto para essa finalidade. A proposta é adequada, pois é a indicação das tecnologias sobre os elementos existentes que caracteriza a passagem do Nível de Contexto para o Nível de Contêiner.
- [ ] 6ª (64): Um integrante afirma que registrar o ADR-003 foi esforço perdido: como a arquitetura provavelmente vai mudar ao longo do semestre, documentar agora uma decisão que pode ser revista é trabalho que será jogado fora assim que a separação de bancos acontecer. Esse argumento não se sustenta: quando a separação ocorrer, um novo ADR registrará a mudança, e o ADR-003 seguirá explicando em que contexto o banco único fez sentido — de modo que o registro atual não é descartado, e sim torna-se o histórico que evita rediscutir a escolha original.
- [ ] 7ª (128): Os defensores dos microsserviços argumentam que, como o próprio ADR-003 admite que a separação pode ser necessária se reservas e catálogo crescerem, o mais eficiente seria já nascer com os serviços separados, evitando o custo de migrar um monólito depois, afinal, refatorar de monólito para microsserviços é sabidamente caro. Esse raciocínio deve ser recusado no contexto dado: antecipar a separação impõe hoje, a uma equipe sem experiência distribuída e com 4 meses, um custo operacional certo para evitar um custo futuro que é apenas possível e condicional ao crescimento.

Resposta (soma): _____

Questão 11 - bônus

Na equipe da ONG **Raízes Urbanas**, quatro pessoas trabalham no mesmo repositório. Durante a Sprint, combinaram que cada funcionalidade seria desenvolvida em um branch próprio e integrada à `main` via Pull Request revisado por outro colega. Dois integrantes preferem fazer commits no fim do dia; os outros dois, a cada poucas horas. Numa dada semana: a integrante A abriu um branch para o cadastro de doações e passou seis dias sem integrá-lo, acumulando muitas mudanças; o integrante B abriu um PR pequeno e

pediu revisão, mas ninguém revisou por três dias; e a dupla C/D, trabalhando no mesmo arquivo em branches diferentes, adiou a integração para o último dia. A `main` está protegida contra push direto. Ao final, houve conflitos grandes e demorados de resolver.

Assinale as verdadeiras:

- ☐ 1ª (2): Considerando que a base da `main` evolui enquanto um branch permanece isolado, o branch de seis dias de A e a integração de última hora de C/D compartilham a mesma causa de fundo: ambos deixaram a distância entre o seu trabalho e a `main` crescer sem reconciliação, de modo que o esforço de merge não foi criado no dia do conflito, mas acumulado silenciosamente nos dias anteriores em que a integração foi adiada.
- ☐ 2ª (4): Como o objetivo de um PR é dar a outro colega a chance de revisar antes da integração, e o PR de B ficou três dias parado sem revisor, a equipe deveria concluir que, quando não há disponibilidade de revisão em tempo hábil, o mais alinhado ao propósito da revisão é autorizar o próprio autor a aprovar e integrar o seu PR, para que o fluxo não trave, a revisão que não acontece a tempo perde a função.
- ☐ 3ª (8): Como dois integrantes commitam só no fim do dia e os outros dois a cada poucas horas, e sabendo que integrações mais frequentes tendem a produzir conflitos menores, a conclusão coerente é que padronizar todos para commitar apenas uma vez ao dia alinharia a cadência da equipe e, com uma única janela diária de integração, concentraria e reduziria os conflitos.
- ☐ 4ª (16): Como o conflito de C/D nasceu de mexerem no mesmo arquivo sem saber um do outro, e como coordenação evita retrabalho, bastaria que a dupla avisasse no chat da equipe, antes de começar, quem vai editar qual arquivo naquele dia; com essa combinação prévia, os dois poderiam seguir em branches separados por vários dias sem que isso gerasse conflito grande na integração final.
- ☐ 5ª (32): Uma vez que a `main` está protegida contra push direto, e que essa proteção obriga toda mudança a passar por um branch e um PR antes de chegar à linha principal, é coerente concluir que foi essa exigência de intermediação que represou as mudanças em branches separados por vários dias, logo, afrouxar a proteção para permitir push direto em casos urgentes reduziria o represamento e, com ele, os conflitos.

Resposta (soma): _____

CIN0136 — Desenvolvimento de Software

Bacharelado em Sistemas de Informação ▪ CIn - UFPE

Profs. Kiev Santos da Gama & Vinicius Cardoso Garcia

✏ Exercício Escolar 1D

13/10/2026

Instruções: Leia as questões com atenção e cuidado, e responda com atenção ao limite de caracteres, quando indicado.

- Esta avaliação é **individual**, **sem consulta**, e vale até **10 pontos** no total. A última questão (Q11) é **bônus** e soma por cima (sua nota pode passar de 10,0).
- A duração total é de **até 2 horas**; organize seu tempo..
- Consulta a qualquer material: **não permitida**.
- Uso de IA ou dispositivos eletrônicos: **não permitido**.
- Leia cada questão com atenção antes de responder. Gerencie bem seu tempo.
- Respostas vagas ou sem justificativa serão consideradas incompletas.
- Dúvidas podem ser expostas, **publicamente**, durante **os primeiros 30 minutos** da realização do Exercício Escolar. **Entender o enunciado** faz parte da avaliação.
- Não é permitido ir ao sanitário durante a realização do exercício, **vá antes**.

Como responder (modelo somatório).

- Cada questão tem um cenário e de 5 a 7 alternativas. Cada alternativa vale um número em potência de 2, conforme sua **posição**: a 1ª vale **2**, a 2ª **4**, a 3ª **8**, a 4ª **16**, a 5ª **32**, a 6ª **64**, a 7ª **128**.
- Marque as alternativas que julgar **verdadeiras** e escreva, no campo de resposta, a **soma** dos valores marcados.
- Correção:
  - Marcou **qualquer alternativa falsa** → a questão vale **0,0** (zera inteira).
  - Marcou **só verdadeiras, mas não todas** → nota proporcional: (nº de corretas marcadas ÷ nº total de corretas) × 1,0.
  - Marcou **todas as corretas e nenhuma falsa** → **1,0**.

Questão 1

A equipe que atende a ONG **Raízes Urbanas** recebeu, de um voluntário que é estudante de programação, um script em Python que gera os relatórios mensais de doações. **A coordenadora pediu que o relatório fosse gerado sob demanda, dentro do próprio servidor Node.js já em produção, toda vez que um usuário clicasse em "gerar relatório" na interface**, ou seja, o código precisaria ser executado como parte do mesmo processo do backend a cada requisição. O script funciona isoladamente na máquina do voluntário. Ao abri-lo, a equipe encontra um único arquivo de 300 linhas, sem testes, sem comentários, com regras de cálculo e acesso ao banco na mesma função. O voluntário se formou e não está mais disponível.

Assinale as verdadeiras:

- [ ] 1ª (2): A dificuldade que a equipe terá para modificar o script, agravada pela saída do voluntário, é um desafio de natureza tanto técnica (código sem estrutura) quanto organizacional (perda de conhecimento e ausência de documentação).
- [ ] 2ª (4): Como a saída do relatório já foi conferida pela coordenadora e está correta, a incorporação pode prosseguir sem testes automatizados por ora, ficando a cobertura de testes como uma melhoria de qualidade desejável, mas não relacionada ao risco de a lógica quebrar em alterações futuras.
- [ ] 3ª (8): Que o script "funcione" e produza o relatório correto é insuficiente para classificá-lo como software de qualidade, porque atributos como manutenibilidade e testabilidade não estão presentes, e são justamente eles que permitirão a evolução do relatório no futuro.
- [ ] 4ª (16): Incorporar o script no estado atual permitiria entregar valor à coordenadora rapidamente; como o comportamento já foi validado na prática, o esforço de reorganizá-lo antes da incorporação seria uma otimização prematura neste momento do projeto.
- [ ] 5ª (32): Rodar um script Python dentro do mesmo processo do servidor Node.js a cada requisição é um obstáculo técnico real de integração, já que os dois runtimes não compartilham o mesmo ambiente de execução, o que torna a diferença de linguagem um problema concreto neste cenário, e não um detalhe secundário.

Resposta (soma): _____

Questão 2

Numa aula de revisão, o professor propõe o seguinte caso. Uma equipe precisa construir um aplicativo de cadastro de beneficiários para uma ONG. A equipe tem cinco integrantes, um dos quais nunca havia trabalhado com banco de dados antes. Duas abordagens são colocadas na mesa. Na **Abordagem 1**, **a equipe entrega na primeira semana uma versão simples do sistema já com os três campos (nome, endereço e histórico), sem validação; nas semanas seguintes, revisita essa mesma versão para adicionar validação e melhorar cada campo com base no uso** (cada entrega aprimora o que já existia). Na **Abordagem 2**, a equipe entrega na primeira semana uma versão do sistema inteiro com os três campos, porém simples, feia e sem validação; nas semanas seguintes, refina essa mesma versão. A coordenadora da ONG só poderá dar feedback a partir da terceira semana, por estar viajando.

Assinale as verdadeiras:

- [ ] 1ª (2): No modelo iterativo, cada ciclo assume explicitamente que a equipe vai aprender algo e que o produto vai mudar, o que torna o feedback, quando disponível, uma entrada planejada do processo, e não uma interrupção dele.
- [ ] 2ª (4): A Abordagem 2 é preferível à Abordagem 1 em qualquer projeto, porque refinar uma versão existente é sempre menos arriscado do que adicionar módulos novos, que podem quebrar o que já funciona.
- [ ] 3ª (8): Ambas as abordagens entregam software ao longo do tempo em vez de tudo no final, o que as diferencia do modelo cascata clássico, no qual a validação do produto pelo usuário tende a ocorrer tardiamente.

- [ ] 4ª (16): O modelo cascata foi um erro histórico da engenharia de software e não possui nenhum contexto de aplicação legítimo nos dias de hoje, tendo sido inteiramente substituído pelos modelos incremental e iterativo.
- [ ] 5ª (32): A Abordagem 1 descreve um modelo incremental, pois o produto cresce por adição de partes novas e completas, enquanto a Abordagem 2 descreve um modelo iterativo.
- [ ] 6ª (64): Como a Abordagem 2 entrega uma versão do sistema inteiro logo na primeira semana, ela corresponde ao modelo cascata, já que o cascata também busca ter o sistema completo o quanto antes.

Resposta (soma): _____

Questão 3

A equipe que atende a ONG **Raízes Urbanas** está encerrando a terceira Sprint. Nesta Sprint, entregaram o módulo de cadastro de voluntários e corrigiram três bugs reportados pela coordenadora na Sprint anterior. Durante a semana houve um desentendimento entre dois integrantes sobre qual editor de código padronizar, o que gerou certo mal-estar. A equipe adotou Scrum com Sprints de duas semanas e usa um board no GitHub Projects. A coordenadora confirmou presença no evento de fechamento da Sprint. **O integrante responsável por garantir que as cerimônias da Sprint aconteçam e por remover os impedimentos que travam a equipe** percebeu que o board acumulou muitos itens simultâneos na coluna "Em progresso".

Assinale as verdadeiras:

- [ ] 1ª (2): O Daily Standup tem como propósito sincronizar a equipe em torno do que foi feito, do que será feito e dos impedimentos, e não substitui a Sprint Review, cujo foco é a inspeção do incremento com o stakeholder.
- [ ] 2ª (4): O acúmulo de muitos itens simultâneos em "Em progresso" contraria a lógica de limitar o trabalho em progresso (WIP); reduzir os itens em andamento tende a diminuir o cycle time, acelerando a entrega efetiva de cada item.
- [ ] 3ª (8): Como a coordenadora estará presente no fechamento da Sprint, a equipe deve aproveitar esse momento para, além de apresentar o incremento, resolver ali o desentendimento sobre o editor de código, usando a presença de uma figura externa para mediar o conflito interno.
- [ ] 4ª (16): O evento de Sprint Review, ao final da Sprint, serve para demonstrar o incremento à coordenadora e coletar seu feedback sobre o que foi construído, enquanto a reflexão da equipe sobre o próprio processo de trabalho pertence à Retrospectiva.
- [ ] 5ª (32): O papel exercido pelo integrante mencionado, garantir as cerimônias e remover impedimentos, corresponde ao do Product Owner, responsável por representar os interesses do stakeholder na ordenação do backlog.
- [ ] 6ª (64): Por usarem um board de visualização de fluxo como o do Kanban, a equipe está, na prática, abrindo mão dos Sprints e do compromisso de entrega ao final de cada ciclo, já que board Kanban e Sprints do Scrum são mutuamente exclusivos.

Resposta (soma): _____

Questão 4

A equipe está levantando requisitos do **PetFood**, o sistema de gestão de produtos para pet shop. O dono mencionou que pretende, no futuro, abrir uma segunda loja em outro bairro. Durante uma visita, a equipe observou a atendente trabalhando: ela consultava os preços num sistema antigo, mas mantinha ao lado um caderno onde anotava quais clientes costumavam comprar ração de marcas específicas, informação que o sistema atual não guardava. A atendente comentou que o caderno era um presente de aniversário. Em conversa posterior, o dono afirmou que **"a busca por um produto tem que retornar o resultado em no máximo 2 segundos, senão a fila no caixa trava"**, e que "qualquer funcionário novo tem que conseguir usar sem treinamento". A equipe também registrou que o sistema deve permitir cadastrar produtos com nome, preço e categoria.

Assinale as verdadeiras:

- ☐ 1ª (2): O caderno indica um requisito que o sistema atual não atende, e o fato de ele ter surgido da observação, e não da fala do dono, ilustra por que restringir a elicitação a perguntar "o que você quer?" tende a produzir um sistema que ignora parte do trabalho real.
- ☐ 2ª (4): A exigência de que "qualquer funcionário novo consiga usar sem treinamento" é um requisito funcional, já que se refere diretamente ao uso do sistema por um usuário.
- ☐ 3ª (8): "Cadastrar produtos com nome, preço e categoria" e "usar o sistema sem treinamento" são requisitos de naturezas distintas: o primeiro descreve uma função verificável por presença ou ausência, o segundo é um atributo de qualidade cuja verificação depende de definir o que conta como "sem treinamento".
- ☐ 4ª (16): A descoberta do caderno pela observação da atendente em seu ambiente real cumpre um papel que a entrevista com o dono dificilmente cumpriria sozinha: expor uma necessidade que ninguém verbaliza porque, para quem trabalha ali, ela já é parte invisível da rotina.
- ☐ 5ª (32): O requisito de desempenho, tal como o dono o expressou, já traz um critério mensurável que permite verificá-lo objetivamente em teste, sem depender de refinamento adicional para saber se foi atendido.

Resposta (soma): _____

Questão 5

A equipe que atende a ONG **Raízes Urbanas** está refinando o backlog antes da sessão com a coordenadora. A reunião foi remarcada duas vezes por causa de um feriado municipal. Entre os itens levantados, aparecem estes três: **(I)** "Como coordenadora, quero gerenciar voluntários, doações, eventos e relatórios para administrar a ONG"; **(II)** "Como voluntária, quero redefinir minha senha para recuperar o acesso à minha conta quando esquecê-la"; **(III) "Como coordenadora, quero receber um alerta quando uma doação estiver próxima da data de validade, para redistribuí-la antes que se perca"**. A equipe

discutiu por dez minutos se usaria Trello ou GitHub Projects para o board. A coordenadora sinalizou que classificaria "quase tudo" como prioridade máxima na hora de priorizar.

Assinale as verdadeiras:

- [ ] 1ª (2): Para o item (II), um critério de aceitação adequado em Gherkin seria: "Dado que a voluntária solicitou a redefinição, Quando ela abre o link após o prazo de validade, Então vê uma mensagem de erro e pode solicitar um novo link", um cenário de caminho triste tão legítimo quanto os de sucesso.
- [ ] 2ª (4): Marcar o item (I) como *Won't Have (now)* significaria descartá-lo definitivamente do produto, razão pela qual essa classificação deve ser evitada para funcionalidades administrativas que a ONG claramente vai precisar.
- [ ] 3ª (8): O item (I) viola o critério *Small* do INVEST e, por reunir quatro áreas administrativas distintas, comporta-se mais como um Epic do que como uma user story, o que sugere quebrá-lo em stories menores antes de estimá-lo.
- [ ] 4ª (16): O item (II) já está pronto como está e não precisa de critérios de aceitação, pois seu benefício ("recuperar o acesso") é claro o suficiente para que a equipe saiba, sem ambiguidade, quando a story foi concluída.
- [ ] 5ª (32): O item (III) segue o formato "Como [persona], quero [ação], para [benefício]" e entrega valor ao usuário, pois seu benefício, evitar a perda de doações por vencimento, é um resultado percebido por quem administra a ONG, e não apenas uma preocupação técnica interna.
- [ ] 6ª (64): Diante de a coordenadora querer classificar "quase tudo" como prioridade máxima, a saída correta em MoSCoW é registrar a maior parte dos itens como *Must Have*, já que *Must* é justamente a categoria que acomoda tudo aquilo que o cliente considera importante.
- [ ] 7ª (128): O item (III) falha no critério *Valuable* do INVEST, pois seu benefício está descrito da perspectiva técnica da equipe, e não como resultado percebido por quem usa o sistema.

Resposta (soma): _____

Questão 6

A equipe está no fim do Sprint 1 do **PetFood**. A funcionalidade de busca de produtos está pronta e passa em todos os critérios de aceitação de funcionalidade que a equipe escreveu. A busca foi implementada por dois integrantes que preferiram programar juntos no mesmo computador. **Ao medir os tempos, a equipe constatou que a busca retorna em menos de meio segundo quando rodada localmente na máquina de desenvolvimento, com a mesma base de milhares de produtos, e só passa a levar cerca de 8 segundos quando acessada pelos computadores do pet shop, cuja conexão de internet oscila muito à tarde.** Além disso, um integrante comentou que o código da busca "funciona, mas está tão embolado que, se precisarem mudar a regra de ordenação depois, vai ser um pesadelo". A equipe vai definir seu Definition of Done antes do Sprint 2.

Assinale as verdadeiras:

- [ ] 1ª (2): Incluir no Definition of Done um critério como "toda funcionalidade nova deve ter o tempo de resposta verificado sob carga realista" operacionaliza qualidade de forma contínua, e o fato de o DoD anterior da equipe não conter nenhum critério de desempenho é justamente o que permitiu que esse problema só aparecesse agora, perto do fim do Sprint.
- [ ] 2ª (4): A melhor forma de garantir qualidade neste caso seria adiar qualquer preocupação com desempenho e manutenibilidade até uma fase final de verificação, quando todas as funcionalidades estiverem prontas e puderem ser avaliadas de uma só vez.
- [ ] 3ª (8): Como a busca retorna em menos de meio segundo localmente, com a mesma base, e só fica lenta quando acessada pela rede oscilante do pet shop, a lentidão de 8 segundos aponta para a infraestrutura externa, e não para um problema de desempenho do próprio sistema de busca.
- [ ] 4ª (16): O código "embolado" que dificultaria mudar a ordenação é um problema de manutenibilidade; e ainda que o usuário final nunca perceba a organização interna do código, isso não torna o problema menos real para a equipe, que arcará com o custo na evolução do sistema.
- [ ] 5ª (32): Uma vez que a busca já passa em todos os critérios de aceitação escritos pela equipe, considerá-la "pronta" é a decisão correta; ajustes de desempenho e organização do código, se necessários, são melhorias que pertencem a um momento posterior à conclusão da funcionalidade.
- [ ] 6ª (64): O fato de a busca passar nos critérios de funcionalidade e ainda assim levar 8 segundos mostra que o próprio sistema de busca falha no atributo de desempenho, independentemente de onde seja executado.

Resposta (soma): _____

Questão 7

Na aula de escopo assistido por IA, a equipe que atende a ONG **Raízes Urbanas** seguiu o roteiro proposto: primeiro, cada integrante escreveu sozinho, sem abrir a IA, o que entendia ser o escopo essencial do **caso de treino PetFood**; só depois compararam entre si e recorreram ao assistente. Um dos integrantes chegou atrasado e teve cinco minutos a menos para escrever sua versão. As versões individuais divergiam bastante. Em seguida, deram à IA o papel de analista cético e pediram que ela apontasse personas esquecidas e o "problema por trás do problema". A sala estava com o ar-condicionado quebrado. **Diferentemente do projeto real, o PetFood é um caso de treino: não há escopo previamente aprovado, e a tarefa da aula é justamente construir o Canvas de Visão e recortar a fronteira do MVP do zero.**

Assinale as verdadeiras:

- [ ] 1ª (2): Ainda que a IA cética tenha feito boas perguntas, isso não equivale a ter elicitado a necessidade real: as lacunas que ela expõe entram no projeto como perguntas a levar à coordenadora, não como requisitos que a equipe já pode dar por levantados.

- [ ] 2ª (4): Uma vez que a IA é boa em revelar personas esquecidas, faz sentido deixar que ela própria decida quais itens entram na fronteira do MVP, contanto que a equipe valide a lista no final.
- [ ] 3ª (8): Manter a divergência individual antes de recorrer à IA foi o que preservou a informação mais útil do exercício: o desacordo entre as visões, que some quando todos partem de uma mesma resposta pronta apresentada pela ferramenta.
- [ ] 4ª (16): O roteiro teria sido mais eficiente se a equipe tivesse aberto a IA primeiro, deixado que ela propusesse uma visão de escopo, e só depois pedido que cada integrante divergisse a partir dela, partindo de uma base estruturada em vez de versões cruas e conflitantes.
- [ ] 5ª (32): Dar à IA o papel de analista cético que questiona antes de gerar, em vez de pedir que ela defina o escopo, protege a equipe contra o viés de confirmação.
- [ ] 6ª (64): Como o PetFood é caso de treino sem escopo aprovado, o uso adequado da IA aqui inclui ajudá-la a estruturar e redigir o Canvas de Visão a partir do zero, cabendo à equipe decidir o conteúdo e cortar o que for genérico.
- [ ] 7ª (128): Se a equipe não consegue explicar, sem o assistente, por que cada história está no backlog e que valor entrega, então, pela política da disciplina, essas histórias ainda não são propriamente da equipe.

Resposta (soma): _____

Questão 8

No **PetFood**, o frontend em React exibe a lista de produtos; o backend em Express expõe a rota `GET /api/produtos`; os dados ficam num PostgreSQL. A equipe hesitou entre `/api/produtos` e `/api/products` e padronizou em português. **Após a última refatoração, o código está separado em camadas: o controller apenas recebe a requisição e monta a resposta HTTP; a regra "só produtos com estoque maior que zero" ficou num service; e todo o acesso ao banco ficou isolado num repository.** Esse integrante é o único da equipe que já tinha mexido com SQL antes. Durante a Sprint Review, a coordenadora pediu que produtos esgotados também aparecessem, marcados como indisponíveis.

Assinale as verdadeiras:

- [ ] 1ª (2): Antes da refatoração, mexer só no formato do JSON obrigaria a abrir também a regra de estoque e o SQL, por estarem todos no mesmo controller, problema que a separação atual em service e repository elimina.
- [ ] 2ª (4): Como o React roda no navegador do usuário e o Express num servidor, os dois estão em processos separados; logo, a regra "estoque maior que zero" poderia ser implementada tanto no React quanto no Express sem diferença arquitetural, já que ambos são código JavaScript da mesma equipe.
- [ ] 3ª (8): Para atender o pedido da coordenadora, a alteração da regra de filtragem agora se concentra no service, deixando o controller e a rota `GET /api/produtos` intocados.

- [ ] 4ª (16): Se o time de frontend precisar exibir os produtos esgotados como indisponíveis, terá de esperar que o backend altere o comando SQL interno, pois é o formato do SQL que o contrato da rota `GET /api/produtos` expõe ao cliente.
- [ ] 5ª (32): O SQL ainda está embutido no controller; para reduzir o acoplamento à tecnologia de persistência, a equipe precisaria, como próximo passo, extrair esse SQL para uma camada de repositório.

Resposta (soma): _____

Questão 9

Na equipe da ONG **Raízes Urbanas**, dois integrantes trabalham no cadastro de voluntários. Os dois combinaram de programar em horários alternados, um de manhã e outro à noite. **Depois da refatoração da semana passada, o antigo `handle()` foi desmembrado: a validação de CPF ficou em `validarCpf.js`, o envio de e-mail em `enviarEmailBoasVindas.js`, o acesso ao banco em `voluntarioRepository.js`, e o `voluntarioController.js` agora só orquestra as chamadas e monta a resposta.** Um dos integrantes usa tema escuro no editor e o outro tema claro. Esta semana, um deles precisou mudar só o texto do e-mail de boas-vindas.

Assinale as verdadeiras:

- [ ] 1ª (2): Extrair a validação de CPF para uma função própria (como o `validarCpf.js` já criado) permite testar se um CPF malformado é rejeitado sem disparar o envio de e-mail nem tocar no banco.
- [ ] 2ª (4): Se os dois integrantes passassem a trabalhar no mesmo turno e a integrar várias vezes ao dia, os conflitos de merge neste arquivo deixariam de ocorrer, pois estariam sempre partindo da versão mais recente um do outro antes de alterar qualquer parte de `handle()`.
- [ ] 3ª (8): Com o envio de e-mail isolado em `enviarEmailBoasVindas.js`, mudar o texto ou trocar o provedor pode ser feito mexendo só nessa parte, sem reabrir a validação nem a resposta.
- [ ] 4ª (16): Renomear `handle()` para `handleCadastroVoluntario()` tornaria a intenção do código mais evidente para quem o lê pela primeira vez e, por deixar o propósito explícito no nome, seria suficiente para resolver a dificuldade que a equipe tem de alterar esse arquivo sem efeitos colaterais.
- [ ] 5ª (32): Quebrar as 250 linhas em dois arquivos de cerca de 125, deixando em cada um um pedaço da validação, um pedaço do e-mail e um pedaço do acesso ao banco misturados, reduziria o número de linhas por arquivo, mas dois desenvolvedores mexendo em responsabilidades distintas ainda poderiam colidir, agora espalhados por dois arquivos em vez de um.
- [ ] 6ª (64): Enquanto a validação, o e-mail, o banco e a resposta compartilharem o corpo da mesma função, dois desenvolvedores mexendo em coisas diferentes tenderão a colidir nas mesmas linhas, situação que ainda descreve o estado atual deste código.

Resposta (soma): _____

Questão 10

Equipe de 20 pessoas distribuídas em vários times, com 3 anos de roadmap e um contrato que já prevê integrar, no primeiro ano, três campi com volume de reservas dez vezes maior que o atual. Hoje o sistema é um monólito Client/Server (React + Express em camadas + um banco PostgreSQL). Na Review, a equipe levou o mesmo diagrama de contexto (Aluno e Bibliotecário usando o Sistema de Reservas, que se conecta ao Sistema de E-mail da Universidade). O ADR-003 registrou a decisão de manter banco único, **mas seu campo de Contexto agora diz: "equipe grande com múltiplos times, crescimento de 10× já contratado para o primeiro ano".**

Assinale as verdadeiras:

- [ ] 1ª (2): Um integrante afirma que registrar o ADR-003 foi esforço perdido: como a arquitetura provavelmente vai mudar ao longo do semestre, documentar agora uma decisão que pode ser revista é trabalho que será jogado fora assim que a separação de bancos acontecer. Esse argumento não se sustenta: quando a separação ocorrer, um novo ADR registrará a mudança, e o ADR-003 seguirá explicando em que contexto o banco único fez sentido, de modo que o registro atual não é descartado, e sim torna-se o histórico que evita rediscutir a escolha original.
- [ ] 2ª (4): Alguém sugere apagar o campo *Consequências* do ADR-003 e deixá-lo só com Contexto e Decisão, sob o argumento de que, como a decisão de banco único claramente foi acertada para o momento, registrar uma possível revisão futura só abre margem para questionamentos desnecessários. Essa edição enfraqueceria o ADR: é justamente o registro do custo e da condição de revisão que impede a equipe futura de tratar a escolha como definitiva e de refazer a análise do zero quando o contexto mudar.
- [ ] 3ª (8): Para a próxima Review, a equipe quer mostrar ao bibliotecário como React, Express e banco se distribuem e se comunicam. Um integrante propõe apenas anotar as tecnologias sobre as caixas do diagrama atual, mantendo os mesmos elementos, e conclui que isso já o converte num diagrama de Contêiner do C4, pronto para essa finalidade. A proposta é adequada, pois é a indicação das tecnologias sobre os elementos existentes que caracteriza a passagem do Nível de Contexto para o Nível de Contêiner.
- [ ] 4ª (16): Diante da insistência nos microsserviços, um integrante propõe encerrar a discussão declarando que o monólito atual é uma escolha "sem trade-offs" para este projeto, já que não se enxerga nenhuma desvantagem nele no prazo de 4 meses, e que isso prova objetivamente que é a decisão certa.
- [ ] 5ª (32): Dois integrantes defendem separar reservas e catálogo em bancos distintos agora, argumentando que o *Bounded Context* prega que cada serviço seja dono do seu dado e que, portanto, o banco único já é uma violação arquitetural a corrigir antes que o acoplamento cresça. Seguir esse argumento seria um erro: o Bounded Context governa *como* delimitar serviços que a equipe decidiu ter, e não é razão, por si, para introduzir a separação de serviços num sistema cujo contexto (4 pessoas, 4 meses) não a justifica.
- [ ] 6ª (64): Como o bibliotecário, na Review, não conseguiu opinar sobre decisões técnicas e ainda perguntou sobre um app de celular fora do escopo, um integrante

conclui que o diagrama de Contexto foi inadequado para aquela audiência e que teria sido melhor levar o diagrama de Contêiner, mais detalhado, para dar a ele base técnica de avaliação.

- [ ] 7ª (128): Os defensores da separação argumentam que já se deveria nascer com serviços separados para evitar o custo de migrar um monólito depois; esse raciocínio deve ser recusado, pois antecipar a separação impõe um custo operacional certo hoje para evitar apenas um custo futuro possível e condicional ao crescimento.

Resposta (soma): _____

Questão 11 - bônus

Na equipe da ONG **Raízes Urbanas**, quatro pessoas trabalham no mesmo repositório, cada funcionalidade em um branch próprio, integrada à `main` via PR revisado. Dois integrantes preferem commits no fim do dia; os outros dois, a cada poucas horas. Numa semana: a integrante A abriu um branch para o cadastro de doações e passou seis dias sem integrá-lo, acumulando muitas mudanças; o integrante B abriu um PR pequeno e pediu revisão, mas ninguém revisou por três dias; **e a dupla C/D, trabalhando no mesmo arquivo em branches diferentes, integrou o trabalho à `main` várias vezes ao dia, resolvendo na hora os pequenos conflitos que surgiam.** A `main` está protegida contra push direto. Ao final, o merge do branch de A foi grande e demorado; o de C/D, tranquilo.

Assinale as verdadeiras:

- [ ] 1ª (2): Como o objetivo de um PR é dar a outro colega a chance de revisar antes da integração, e o PR de B ficou três dias parado sem revisor, a equipe deveria concluir que, quando não há disponibilidade de revisão em tempo hábil, o mais alinhado ao propósito da revisão é autorizar o próprio autor a aprovar e integrar o seu PR, para que o fluxo não trave, a revisão que não acontece a tempo perde a função.
- [ ] 2ª (4): O branch longo de A e o de C/D compartilham a mesma causa de fundo: ambos deixaram a distância até a `main` crescer sem reconciliação, acumulando o esforço de merge ao longo dos dias.
- [ ] 3ª (8): O contraste entre o merge demorado de A e o tranquilo de C/D, no mesmo repositório e sob as mesmas regras, indica que o fator decisivo do custo do merge não foi usar branch nem tocar arquivos compartilhados, mas a frequência com que cada um reconciliou seu trabalho com a `main` ao longo da semana.
- [ ] 4ª (16): Uma vez que a `main` está protegida contra push direto, e que essa proteção obriga toda mudança a passar por um branch e um PR antes de chegar à linha principal, é coerente concluir que foi essa exigência de intermediação que represou as mudanças em branches separados por vários dias, logo, afrouxar a proteção para permitir push direto em casos urgentes reduziria o represamento e, com ele, os conflitos.
- [ ] 5ª (32): Já que o conflito concreto surgiu entre C e D por editarem o mesmo arquivo em paralelo, e como o Git resolve automaticamente merges quando as pessoas trabalham em arquivos diferentes, a lição consistente com o funcionamento do Git é organizar a divisão de tarefas de modo que dois integrantes nunca fiquem

responsáveis por um mesmo arquivo na mesma Sprint, eliminando na origem a possibilidade de conflito.

Resposta (soma): _____